
Capacity-Agnostic Parameter Isolation for Continual Graph Learning

Ye Xiao^{* 1} Ruikun Li^{* 2} Zhenyu Yang³ Andrey Vasnev¹ Junbin Gao¹

Abstract

Existing parameter isolation-based continual learning methods employ diverse designs to accommodate more tasks within limited model capacity, but often incur increasing computational overhead as model capacity expands for growing task streams. To address this efficiency bottleneck, we propose CAGNN, a graph continual learning framework with a biological neuron-inspired architecture that features capacity-agnostic efficiency. CAGNN leverages graph contextual information to construct task-specific subnetworks and decouples them during training and inference, reducing full-network propagation overhead while enabling knowledge transfer across tasks. Extensive experiments demonstrate CAGNN’s superior effectiveness and computational efficiency over state-of-the-art methods. Our code is available at: <https://github.com/WonderHeiYi/CAGNN>.

1. Introduction

“Continual learning is the constant development of complex behaviors with no final end in mind (Ring, 1994)”. As a popular continual learning paradigm, parameter isolation-based methods inevitably enlarge model capacity when parameter usage becomes saturated, incurring substantial space and computational overhead, especially on large datasets (Yuan et al., 2026; Zhang et al., 2022). Although existing methods improve the utilization efficiency of the available parameter budget (Wang et al., 2024), they largely overlook the computational complexity caused by capacity expansion. Specifically, as shown in Figure 1 (Left), they rely on *full-network propagation*, where task-specific computations are still performed over the entire shared network, so many unused, i.e.,

masked, parameters participate in propagation. With disjoint subnetworks and D_{out} output units per task, the model’s per-task complexity will become $\mathcal{O}(D_{in} \cdot (TD_{out}))$, increasing linearly with the expected maximum task number T . Under the assumption that continual learning has no final end, such methods eventually face a severe efficiency bottleneck. Therefore, “*improving the computational efficiency for the parameter isolation-based method under model capacity expansion*” becomes a critical demand for this paradigm.

A straightforward solution is to allocate a neatly partitioned parameter block to each task (e.g., neuron-wise selection). However, it induces coarse-grained parameter reuse that limits parameter reusability and can bias knowledge transfer (Mallya & Lazebnik, 2018; Golkar et al., 2019; Zhang et al., 2023; Ko et al., 2025). Parameter reusability is a critical metric of interest; although reducing reuse may avoid bias, it rapidly exhausts free parameters. Fine-grained methods such as Piggyback (Mallya et al., 2018), SupSup (Golkar et al., 2019), and WSN (Kang et al., 2024) instead learn self-adaptive masks to support element-wise knowledge transfer. Yet, these masks entail continual optimization, lack clear interpretability for parameter reuse, and, due to their irregularity, still incur General Matrix-Matrix Multiplication operations on the full network. Ultimately, it turns back to the original challenge of pursuing efficiency.

Inspired by biological neurons that possess distal dendrites to perceive contextual stimuli and adapt to diverse environments (Schachter et al., 2010; Harnett et al., 2012; Li et al., 2025) and the underexplored role of parameter isolation in the graph domain (Zhang et al., 2024b; Ko et al., 2025), we propose the Capacity-Agnostic Graph Neural Network (CAGNN) for efficient continual learning with flexible reusability. CAGNN achieves twofold efficiency through a Context-aware Hierarchical Masking (CHM): (1) CHM leverages task-specific context as a prior to construct subnetworks during a brief warm-up, where parameters are shared based on cross-task common patterns, thereby avoiding persistent mask optimization and improving the interpretability of parameter selection; (2) By design, CHM supports memory-friendly mask storage and enables training of each task in a decoupled fashion, where every task-specific subnetwork is structurally compressible and can be decoupled from the holistic network (Figure 1 Right). After warm-up, CAGNN trains and infers as a compact vanilla backbone,

^{*}Equal contribution ¹Business School, University of Sydney, Sydney, Australia ²Financial Technology Thrust, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China ³School of Computing, Macquarie University, Sydney, Australia. Correspondence to: Ye Xiao <ye.xiao@sydney.edu.au>.

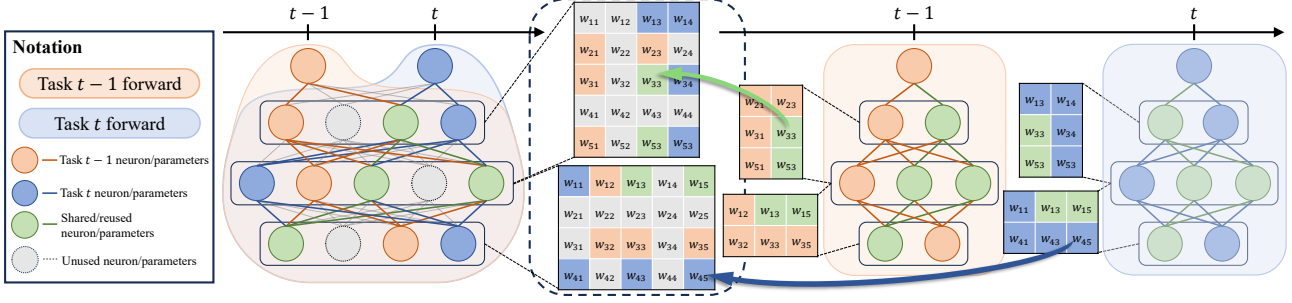


Figure 1. Intuitive comparison of full-network and decoupled propagation in parameter-isolation continual learning. Conventional methods, such as PackNet (Mallya & Lazechnik, 2018), Piggyback (Mallya et al., 2018), HAT (Serra et al., 2018), and WSN (Kang et al., 2024), propagate through the entire network despite using only task-specific subnetworks (Left). Our method instead decouples selected subnetworks from unselected parameters while preserving a shared parameter space for knowledge transfer (Right). TAAM (Liu & Zhang, 2026) provides a related alternative by assigning task-specific modulators to a frozen backbone, but it isolates new knowledge into separate modulators and only promotes coarse-grained parameter sharing on the entire frozen backbone.

excluding unused parts from other tasks.

We theoretically justify parameter reuse under CHM and introduce a Fisher information-based regularization (Kirkpatrick et al., 2017; Liu et al., 2021) to refine subnetwork structures during warm-up. These designs collectively reduce the sensitivity to the model’s capacity, alleviating computational complexity brought by capacity expansion. To the best of our knowledge, this is the first work to explicitly study capacity-driven efficiency degradation in parameter isolation under continual model capacity expansion. Empirical analysis verifies that graph contexts provide effective initial bias for subnetwork generation, and comprehensive experiments show that CAGNN well-handles catastrophic forgetting and improves computational efficiency on node- and graph-level task-incremental learning benchmarks. The contributions are summarized as follows:

- We identify the efficiency bottleneck in parameter isolation-based continual learning induced by capacity expansion, and highlight computational efficiency as a crucial dimension of performance for the parameter isolation-based paradigm.
- We propose a novel biological neurons-inspired GNN framework, which possesses architectural superiority that leverages the task-specific context for efficient subnetwork construction and decouples task-specific subnetworks for capacity-agnostic efficient training and inference.
- We provide a theoretical justification for the proposed architecture and introduce a Fisher information-based regularization to promote effective parameter reuse for greater model scalability for the ongoing task stream.

2. Preliminaries

This section first introduces the dendritic neuron model and reformulates it into a generalized structure, which underpins

the proposed CAGNN framework. We then formalize continual graph learning under the parameter-isolation paradigm.

2.1. Dendritic Neuron Model

A neuron’s soma itself provides a limited membrane area for synaptic input, constraining its responsiveness to external stimuli. However, a neuron’s dendrites enlarge the receptive surface and actively transform synaptic signals, enabling the integration of diverse contextual stimuli and modulation of somatic output (Fiala & Harris, 1999; Harnett et al., 2012; Schachter et al., 2010). This mechanism has inspired various dendritic neuron models (Gao et al., 2018; 2021; Rall, 1962; Todo et al., 2019). Following previous studies (Grewal et al., 2021; Wu et al., 2018a; Iyer et al., 2022; Li et al., 2025), the dendritic neuron model is formulated as follows:

$$\hat{y} = (\mathbf{W}_f^\top \mathbf{x} + \mathbf{b}) \odot \sigma \left(\max_{b \in B} (\mathbf{W}_d \odot \mathbf{M}_d)^\top \mathbf{c} \right) \quad (1)$$

The model is a joint of two units. The former feedforward unit acts as the soma and computes the base response from input $\mathbf{x}$. The latter dendritic unit perceives the context $\mathbf{c}$ to generate branch-wise modulation signals. The binary mask $\mathbf{M}_d$ determines branch b ’s connection with each dendrite input unit, across B branches in each output unit without replacement, to enforce synaptic connectivity in dendrite weight $\mathbf{W}_d$. The strongest branch response of each dendrite output is selected and passed through a sigmoid gate $\sigma(\cdot)$ to modulate the feedforward output via Hadamard product $\odot$.

2.2. Problem Definition

We study task-incremental continual graph learning under the parameter-isolation paradigm. Denoting a graph as $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{X}\}$, where $\mathbf{X} \in \mathbb{R}^{N \times F}$ denotes node features. Given a sequence of class-disjoint tasks $\mathcal{T} = \{\mathcal{T}_1, \dots, \mathcal{T}_T\}$, the model learns tasks sequentially, with task identity available during both training and inference. For each task $\mathcal{T}_t$, parameter isolation learns a binary mask $\mathbf{m}_{\mathcal{T}_t}$ to select a

Capacity-Agnostic Parameter Isolation for Continual Graph Learning

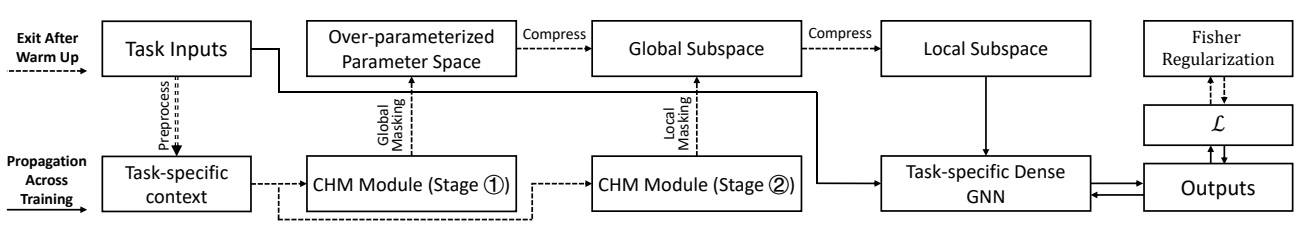


Figure 2. A brief demonstration of CAGNN, consisting of a graph convolutional feedforward part and a masking part. The graph convolutional feedforward part operates across the entire training to learn representations, while the masking part searches subnetwork structures during the warm-up phase. When the warm-up phase is completed, the subnetwork structure is fixed for the corresponding task.

task-specific subnetwork from the shared parameter space θ , i.e., $\theta_{\mathcal{T}_t} = \mathbf{m}_{\mathcal{T}_t}(\theta)$. After task $\mathcal{T}_t$ is learned, its training data is no longer accessible.

Node Classification $\mathcal{T}_t$ contains a subgraph $\mathcal{G}_t$ with labeled nodes $\mathcal{V}'_t$, and a graph operator $f_\theta : \mathcal{G} \mapsto \mathbb{R}^{N \times C}$ is optimized by:

$$\theta_{\mathcal{T}_t}^*, \mathbf{m}_{\mathcal{T}_t}^* = \arg \min_{\theta, \mathbf{m}_{\mathcal{T}_t}} \frac{1}{|\mathcal{V}'_t|} \sum_{v \in \mathcal{V}'_t} \mathcal{L}(f(\mathcal{G}_t; \mathbf{m}_{\mathcal{T}_t}(\theta))[v], y_v)$$

Graph Classification $\mathcal{T}_t$ contains a labeled graph set $G_t = \{(\mathcal{G}_i, y_i)\}_{i=1}^{n_t}$, and $f_\theta : \mathcal{G} \mapsto \mathbb{R}^{1 \times C}$ is optimized by:

$$\theta_{\mathcal{T}_t}^*, \mathbf{m}_{\mathcal{T}_t}^* = \arg \min_{\theta, \mathbf{m}_{\mathcal{T}_t}} \frac{1}{|G_t|} \sum_{G_i \in G_t} \mathcal{L}(f(G_i; \mathbf{m}_{\mathcal{T}_t}(\theta)), y_i)$$

3. Methodology

In this section, we present the proposed Capacity-Agnostic Graph Neural Network (CAGNN) and its efficiency-oriented design for continual graph learning. Simulating real neurons that perceive external synaptic stimulation and modulate somatic signals through spike activation via their distal dendrites, CAGNN adopts an over-parameterized Graph Convolutional Networks (GCN) (Kipf, 2017) as the feedforward unit and employs Context-aware Hierarchical Masking (CHM) that activates or inhibits the model’s weights to construct task-specific subnetworks. As shown in Figure 2, when a new task $\mathcal{T}_t$ arrives, CHM exploits task-specific context to perform global-to-local masking, compressing the over-parameterized parameter space into a dense task-specific subspace. For $t > 1$, Fisher information-based regularization further refines the subnetwork structure for effective parameter reuse. After a brief warm-up, CHM is deactivated, and the selected subnetwork structure is frozen for the remaining training. Afterward, CAGNN continues training as a lightweight conventional GCN within the selected subspace. The algorithm is provided in Appendix A.

3.1. Context-Aware Hierarchical Masking

The Context-aware Hierarchical Masking (CHM) module uses contextual information as prior knowledge to construct structured task-specific subnetworks through two stages,

which are dendritic masking and low-rank masking. Together, they form a hierarchical scheme that enables fine-grained parameter selection.

3.1.1. DENDRITIC MASKING (STAGE 1)

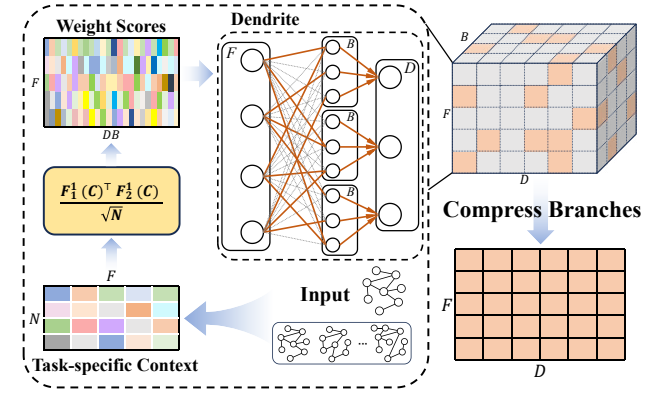


Figure 3. Illustration of Dendritic Masking.

Unlike dense activation in conventional deep neural networks, dendritic activation is characterized by sparse synaptic connectivity, where each dendritic unit randomly forms branch connections from the input without replacement. This branch-wise activation induces *intragroup sparsity* (Wu et al., 2018a), which was originally introduced to improve data locality and inference efficiency compared to irregular element-wise sparsity. We further observe that intragroup sparsity offers an additional benefit for subnetwork construction, which is capable of capturing richer patterns from contextual inputs by uniformly involving all input channels. By further exploiting the dendritic unit’s structural regularity, we propose dendritic masking, which naturally supports network compression that facilitates efficient training and inference.

The dendritic masking for task $\mathcal{T}_t$ is visually shown in Figure 3. To be specific, we initialize a weight $\mathbf{W}_0$ with a dendritic shape $F \times D \times B$, where F , D , and B , respectively, are the input dimension, the intermediate dimension, and the number of branches. Given $\mathcal{T}_t$ ’s context matrix $\mathbf{C} \in \mathbb{R}^{N \times F}$ as input, where N is the number of samples, we compute the weight scores $\mathcal{S}_{global}$ by self-attention:

$$\mathcal{S}_{global} = \frac{F_1(\mathbf{C})^\top F_2(\mathbf{C})}{\sqrt{N}} \in \mathbb{R}^{F \times DB} \quad (2)$$

where $F_1(\cdot) : \mathcal{C} \mapsto \mathbb{R}^{N \times F}$ and $F_2(\cdot) : \mathcal{C} \mapsto \mathbb{R}^{N \times DB}$ are two linear projections. Afterward, $\mathcal{S}_{global}$ is reshaped into a 3D tensor of the branch connection scores as $\mathcal{S} = [\mathcal{S}(f, d, b)] \in \mathbb{R}^{F \times D \times B}$ where $\mathcal{S}(f, d, b)$ denotes the score at position (f, d, b) of $\mathbf{W}_0$ ($1 \leq f \leq F$, $1 \leq d \leq D$ and $1 \leq b \leq B$).

Following Eq. (1), we define our dendritic activation $\Phi(\mathbf{W}_0|\mathcal{S})$ by taking weights with a maximum score along the third tensor dimension, i.e., gathering the most activated branch for each position (f, d) . This map $\Phi : \mathbb{R}^{F \times D \times B} \mapsto \mathbb{R}^{F \times D}$, however, is nondifferentiable due to the max-pooling. To enable end-to-end optimization, we decompose this activation into a masking operation and approximate it with the straight-through estimation trick (STE) (Bengio et al., 2013). For this purpose, we introduce the dendritic mask $\mathbf{M}_{dm} = [\mathbf{M}_{dm}(f, d, b)] \in \{0, 1\}^{F \times D \times B}$ for each input-intermediate pair (f, d) in the following way:

$$\mathbf{M}_{dm}(f, d, b) = \begin{cases} 1 & \text{if } b = \tilde{b} \\ 0 & \text{if } b \neq \tilde{b} \end{cases} \quad (3)$$

where $\tilde{b} = \arg \max_b \{\mathcal{S}(f, d, b)\}$. $\mathbf{M}_{dm}$ only activates the weight with the highest score learned from $\mathcal{C}$ among the B branch connections of each input, resembling sparse synaptic connectivity.

In practice, we sample the binary dendritic mask $\mathbf{M}_{dm}$ along the branch dimension by employing Gumbel-Softmax (Jang et al., 2016; Maddison et al., 2016) with STE, which approximates discrete masking in a differentiable manner. STE guarantees the gradient flow during propagation, thus rendering $\mathcal{S}_{global}$, together with the model parameter $\mathbf{W}_0$, optimizable. Specifically:

$$\mathbf{M}_{dm} = \text{Softmax}_b((\log(\mathcal{S}) - \log(1 - \mathcal{S}) + g)/\tau) \quad (4)$$

where $g \in \mathbb{R}^{F \times D \times B}$ denotes Gumbel noise sampled from $\text{Gumbel}(0, 1)$, and τ controls the sharpness of Gumbel-Softmax that the output approaches one-hot vectors as $\tau \rightarrow 0$ and a uniform distribution as $\tau \rightarrow \infty$.

To compress model's parameters $\mathbf{W}_0$ into a dense subspace $\mathbf{W}_{global}$, we gather selected activated branches by applying $\mathbf{M}_{dm}$ on $\mathbf{W}_0$ using the Hadamard product, with a summation trick on the outputs across branches of each dendritic unit to preserve the gradient of each masking score. Ultimately, we have the following relation:

$$\mathbf{W}_{global} = \Phi(\mathbf{W}_0|\mathcal{S}) = \text{sum}_3(\mathbf{M}_{dm} \odot \mathbf{W}_0) \quad (5)$$

where $\text{sum}_3(\cdot)$ denotes sum pooling along the branch dimension B . By design, dendritic masking thereby partitions task-relevant parameters into branch-wise regions based on task-specific context, reducing cross-task interference while encouraging a global-level parameter reuse across tasks.

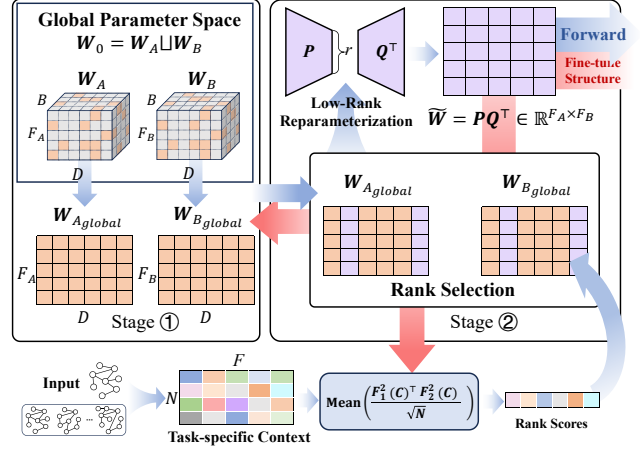


Figure 4. Illustration of Low-rank Masking.

3.1.2. LOW-RANK MASKING (STAGE 2)

To improve the model's scalability, task-specific subnetworks should be highly sparse within the global parameter space. Yet dendritic masking alone only yields $1/B\%$ sparsity, which constrains the shape over the branch dimension and subnetworks' flexibility. However, under the same model's capacity condition, simply increasing B narrows the hidden width and may incur an information bottleneck, especially in GNNs with the over-squashing problem (Alon & Yahav, 2021). Keeping in mind the objective of computational efficiency, to pursue a further high granularity-level of parameter masking and sharing, we therefore keep B moderate and introduce low-rank masking as the second stage for finer-grained sparsity, as shown in Figure 4.

Low-rank methods, such as the low-rank approximation and the low-rank adaptation, have been widely studied in large-scale models for efficiency, and have been proven that low intrinsic-rank weights usually suffice or even bring better generalization than full-rank counterparts do (Denton et al., 2014; Hu et al., 2022; Li et al., 2018; Aghajanyan et al., 2021; Gunasekar et al., 2017). Our low-rank masking begins with an over-parameterized parameter space $\mathbf{W}_0$ separated into two complementary spaces $\mathbf{W}_A \in \mathbb{R}^{F_A \times D \times B}$ and $\mathbf{W}_B \in \mathbb{R}^{F_B \times D \times B}$ and then two dense subspaces $\mathbf{W}_{A_{global}} \in \mathbb{R}^{F_A \times D}$ and $\mathbf{W}_{B_{global}} \in \mathbb{R}^{F_B \times D}$ derived by the dendritic masking. Subsequently, we use self-attention scoring to obtain a score $\mathcal{S}_{local} \in \mathbb{R}^{F \times D}$, which is averaged to rank scores $s_{local} \in \mathbb{R}^{1 \times D}$ for rank selection. Different from neuron-wise masking, low-rank masking selects top- r columns of $\mathbf{W}_{A_{global}}$ and $\mathbf{W}_{B_{global}}$, and yields two low-rank factors $\mathbf{P} \in \mathbb{R}^{F_A \times r}$ and $\mathbf{Q} \in \mathbb{R}^{F_B \times r}$, forming a local subspace. Note that r is determined by the target sparsity. Specifically:

$$\mathbf{P} = \mathbf{W}_{A_{global}}[:, \mathcal{I}], \quad \mathbf{Q} = \mathbf{W}_{B_{global}}[:, \mathcal{I}] \quad (6)$$

where $\mathcal{I} = \text{Top}_r(s_{local})$ are selected columns. Ultimately, we form the forward pass weight $\tilde{\mathbf{W}}$ through low-rank repara-

parameterization:

$$\tilde{W} = PQ^\top \quad (7)$$

and express the forward pass of the layer l of CAGNN as:

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} \tilde{W}^{(l)}) \quad (8)$$

where H denotes the node representation matrix, initialized as X in the first layer, and $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ denotes the normalized adjacency matrix.

Rank selection truncates the gradient flow and introduces differentiability issues. Directly applying STE still encounters convergence difficulty on the subnetwork structure because the selection operation is inherently non-smooth. To preserve the adaptivity of low-rank masking, we reformulate rank selection as dendritic sparse synaptic connectivity and create a binary dendritic mask $M_{\mathcal{I}}$ as a differentiable proxy for $\mathcal{I}$. Specifically, given r columns to be selected, we reshape $s_{local} \in \mathbb{R}^{1 \times D}$ into $\mathbb{R}^{1 \times r \times k}$, where $k = D/r$. Following Eq. (4), we obtain $M_{\mathcal{I}} \in \mathbb{R}^{1 \times r \times k}$ via Gumbel sampling, which partitions s_{local} into k groups and performs group-wise column activation. Accordingly, we reshape $W_{A_{global}} \in \mathbb{R}^{F_A \times D}$ and $W_{B_{global}} \in \mathbb{R}^{F_B \times D}$ into $\mathbb{R}^{F_A \times r \times k}$ and $\mathbb{R}^{F_B \times r \times k}$, respectively, and formulate P_{soft} and Q_{soft} via the summation trick as Eq. (5), preserving the gradient flow to each masking score:

$$\begin{aligned} P_{soft} &= \text{sum}_3(M_{\mathcal{I}} \odot W_{A_{global}}) \in \mathbb{R}^{F_A \times r} \\ Q_{soft} &= \text{sum}_3(M_{\mathcal{I}} \odot W_{B_{global}}) \in \mathbb{R}^{F_B \times r} \end{aligned} \quad (9)$$

Note that in practice, we temporarily increase r until $r \mid D$, and prune the selected columns back to the target rank after warm-up. During warm-up, the forward pass weight is consistently constructed by $\tilde{W}_{soft} = P_{soft} Q_{soft}^\top$, which acts as a differentiable proxy of $\tilde{W}$ for subnetwork structure learning. The mask $M_{\mathcal{I}}$ is then mapped back to W_0 , also in a differentiable manner, to obtain the global task-specific mask M_{CHM} for Fisher regularization.

Integrating the above steps into the CHM operation $\mathbb{M}_{CHM}$, the warm-up forward pass of CAGNN incorporating dendritic activation into the feedforward unit is formulated as:

$$H^{(l+1)} = \sigma(\hat{A} H^{(l)} \mathbb{M}_{CHM}(W_0, C)) \quad (10)$$

When the warm-up is completed, CHM fixes the subnetwork by recording M_{dm} and $\mathcal{I}$ to reconstruct $\tilde{W}$ from W_0 , forming a general depiction of the subnetwork's structure. Due to the hierarchical design, these masks can be stored as indices and used for reconstruction via indexing, requiring much less memory than dense masks. Afterward, $\mathbb{M}_{CHM}(W_0, C)$ is fixed as $\tilde{W}$ following Eqs. (5)–(7), allowing CAGNN to train and infer equivalently on a dense, lightweight conventional GCN as in Eq. (8). Throughout the training, an accumulated mask M_{accum} freezes the parameters used by previous tasks and is updated with the final M_{CHM} after training each task.

3.2. Task-specific Context

Task-specific context plays a crucial role in subnetwork construction, as it is expected to encapsulate compact yet informative representations that capture both task distinguishability and cross-task transferability.

Node Classification We construct the task-specific context C_{node} by one-hop aggregation on $\mathcal{G}_t$:

$$C_{node} = \hat{A}_t X_t \quad (11)$$

This corresponds to a low-pass filter that smooths the node features while relatively depicting $\mathcal{G}_t$ in raw (Nt & Maehara, 2019; Wu et al., 2019).

Graph Classification We adopt the Rayleigh Quotient distribution (Chan et al., 2011) to capture class-consistent patterns across graphs with varying sizes (Dong et al., 2024). The graph-level context C_{graph} is defined by evaluating the Rayleigh Quotient $\mathcal{R}_i$ for each graph $\mathcal{G}_i \in \mathcal{G}_t$:

$$C_{graph} = \{\mathcal{R}_i\}_{i=1}^n \in \mathbb{R}^{n \times F} \quad (12)$$

where $\mathcal{R}_i = \text{diag}\left((X_i^\top L_i X_i) / (X_i^\top X_i)\right)$, and $L_i = I - \hat{A}_i$ is the Laplacian matrix of $\mathcal{G}_i$.

3.3. Theoretical Analysis

We theoretically justify the robustness of low-rank masking to cross-task reuse bias and introduce Fisher regularization to promote effective parameter reuse. Detailed proofs are provided in Appendix C. We also provide a brief complexity analysis, with a further detailed discussion in Appendix D.

3.3.1. EFFECTIVE PARAMETER REUSE

Notation. Considering a certain level of parameter reuse in the continual learning problem, we write the masked forward-pass weight as $W = W^f + W^t \in \mathbb{R}^{F \times D}$, where W^f denotes the frozen reused weights from previous tasks and W^t denotes the trainable weights for the current task. Given the unknown task-optimal weight W^* , we define the reuse-induced bias as $\Delta := W^* - W^f$.

Proposition 3.1 (Reuse Bias under Neuron-wise Selection/Masking). *Let columns of the weight be partitioned into a frozen set $\mathcal{I}_f$ and a trainable set $\mathcal{I}_t = \mathcal{I} \setminus \mathcal{I}_f$. The minimum parameter bias from the optimal weight W^* brought by neuron-wise masking with pure column-wise reuse is limited by an irreducible bias from the not learnable portion $W^f = W_{:, \mathcal{I}_f}$, such that:*

$$\inf_{W_{:, \mathcal{I}_t}} \|W^* - W\|_F = \|\Delta_{:, \mathcal{I}_f}\|_F \quad (13)$$

Theorem 3.2 (Reuse Bias under Low-rank Masking). *Let $W = PQ^\top$ be with $\text{rank}(W) \leq r$. The columns of P*

and $\mathbf{Q}$ are partitioned into a frozen set $\mathcal{I}_f$ and a trainable set $\mathcal{I}_t = \mathcal{I} \setminus \mathcal{I}_f$, where $r_t = |\mathcal{I}_t| < r$ denotes the effective trainable rank. Under low-rank masking with column-wise reuse, the minimum reuse-induced bias depends only on the effective trainable rank r_t :

$$\inf_{\text{rank}(\mathbf{W}^t)} \|\mathbf{W}^* - \mathbf{W}\|_F = \|\Delta - \Delta_{(r_t)}\|_F = \left(\sum_{i>r_t}^p \sigma_i^2 \right)^{\frac{1}{2}} \quad (14)$$

where the infimum is taken over the trainable component $\mathbf{W}^t$ with rank at most r_t , $\Delta = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ is the SVD of the reuse bias with singular values $\mathbf{\Sigma} = \text{diag}(\sigma_1 \geq \dots \geq \sigma_p \geq 0)$, and $\Delta_{(r_t)} := \mathbf{U}\mathbf{\Sigma}_{(r_t)}\mathbf{V}^\top$ is its best rank- r_t approximation.

With the same number of reused and trainable parameters, low-rank masking achieves a smaller minimum reuse bias than neuron-wise masking when Δ is well captured by a rank- r_t subspace. In particular, if $\text{rank}(\Delta) \leq r_t$ and $\Delta_{\mathcal{I}_f} \neq 0$, the error, which is contributed by tail singular values of Δ , vanishes (i.e., $\sum_{i>r_t}^p \sigma_i^2 \approx 0$), whereas neuron-wise masking still retains the irreducible bias $\|\Delta_{\mathcal{I}_f}\|_F$. Thus, between two masking schemes designed for efficiency, low-rank masking offers greater flexibility in absorbing cross-task bias under the same parameter budget.

3.3.2. FISHER REGULARIZATION

Since the task-optimal solution is unknown a priori, quantifying whether reused parameters facilitate knowledge transfer is usually infeasible without a fully flexible reference run. We therefore establish the following relationship between parameter reuse and model loss:

Proposition 3.3. *Given the optimal parameter θ^* for task $\mathcal{T}_t$, the excess loss $\Delta\mathcal{L} = \mathcal{L}(\theta) - \mathcal{L}(\theta^*)$ can be expressed by:*

$$\Delta(\mathcal{L}) \approx \frac{1}{2} \Delta\theta^\top \mathbf{H}(\theta^*) \Delta\theta + o(\|\Delta\theta\|^2) \quad (15)$$

where $\mathbf{H}(\theta^*)$ denotes the Hessian matrix of the loss at θ^* .

Corollary 3.4. *The variation is governed by δ since $\mathbf{H}(\theta^*)$ is a fixed constant. If r_t is sufficient, $\Delta\theta = 0$, within which the reused part θ_f itself is already optimum. Otherwise, any unavoidable deviation $\delta_f = \theta_f - \theta_f^*$ leads to $\Delta\theta \neq 0$.*

To mitigate the impact of reuse when r_t is insufficient, we utilize the diagonal of the Fisher information matrix $\mathcal{F}$ (Aich, 2021; Kirkpatrick et al., 2017) as a proxy of δ_f and introduce a regularization term by minimizing:

$$\mathbf{M}_{\text{CHM}}^* = \arg \min_{\mathbf{M}_{\text{CHM}}} \frac{1}{|\theta_f|} \sum_{i=1}^{|\theta|} \mathcal{F}_{\theta_f, i} \quad (16)$$

where $\mathcal{F}_{\theta_f} = (\mathbf{M}_{\text{CHM}} \odot \mathbf{M}_{\text{accum}; \mathcal{T}_{t-1}}) \odot \mathcal{F}_\theta$, and $\mathbf{M}_{\text{CHM}} \odot \mathbf{M}_{\text{accum}; \mathcal{T}_{t-1}}$ indicates reused parameters. Minimizing this term encourages CHM to reuse parameters with

lower Fisher values, i.e., parameters which are close to optimum θ_f^* or to which the current-task loss is less sensitive. Equivalently, it forces $\theta_f \rightarrow \theta_f^*$ and thereby reduces the deviation from θ_f^* .

3.3.3. COMPLEXITY ANALYSIS

We analyze the warm-up complexity of CAGNN against WSN (Kang et al., 2024) under a matched parameter budget, with N nodes, $|\mathcal{E}|$ edges, hidden dimension F , branch number B , and selected rank $r = (1 - p)BF$, where p is the sparsity level. Under the same capacity, WSN has a hidden dimension $F_w = \sqrt{2BF}$. During warm-up, WSN performs mask selection and full-network propagation over the entire parameter space, leading to the complexity:

$$\mathcal{O}(2BF^2 + 2BNF^2 + \sqrt{2B}|\mathcal{E}|F) \quad (17)$$

In contrast, CAGNN learns masks through CHM and performs low-rank propagation with complexity:

$$\mathcal{O}(NF^2 + NFr + |\mathcal{E}|F) \quad (18)$$

When a pooled task context vector is used, the mask-learning cost is further reduced, yielding:

$$\mathcal{O}(F^2 + NFr + |\mathcal{E}|F) \quad (19)$$

Since $r \ll F_w$ in practice, CAGNN avoids warm-up full-network propagation and achieves lower computational cost, especially as the model capacity grows.

4. Experimental Evaluation

We evaluate CAGNN on eight real-world datasets under the task-incremental setting, examining its ability to mitigate catastrophic forgetting as well as the efficiency bottleneck largely overlooked by existing parameter isolation methods.

4.1. Experimental Setup

We briefly introduce the datasets and baselines below, with full experimental details provided in Appendix E.

Datasets We evaluate node classification on five datasets, including Cora (Sen et al., 2008), Citeseer (Sen et al., 2008), CoraFull (Bojchevski & Günnemann, 2018), OGB-Arxiv (Hu et al., 2020), and Amazon Computers (McAuley et al., 2015). We further evaluate graph classification on three datasets, including MNIST (Achanta et al., 2012), CIFAR10 (Dwivedi et al., 2023), and Aromaticity (Wu et al., 2018b).

Baselines We mainly evaluate CAGNN against parameter isolation-based methods, including PackNet (Mallya & Lazebnik, 2018), Piggyback (Mallya et al., 2018), HAT (Serra et al., 2018), TAAM (Liu & Zhang, 2026), and

Table 1. Performance of CAGNN on GCN (CAGCN) and GAT (CAGAT), and baselines on 8 datasets. Results are reported as the Average Performance (AP) and Average Forgetting (AF) of 5 iterations. Each dataset’s best and second-best results are highlighted with **gray** and underline, respectively. OOM indicates out-of-memory during training, and N/A indicates not applicable.

DATASET (# Task)	NODE CLASSIFICATION								GRAPH CLASSIFICATION							
	CORA (3)		CITSEER (3)		CORAFULL (35)		COMPUTERS (5)		OGB-ARXIV (8)		MNIST (5)		CIFAR10 (5)		AROMATICITY (10)	
	AP	AF	AP	AF	AP	AF	AP	AF	AP	AF	AP	AF	AP	AF	AP	AF
Bare	90.28 ± 2.2	2.1 ± 1.7	80.43 ± 3.6	5.3 ± 1.9	79.79 ± 3.5	19.8 ± 4.2	96.18 ± 1.9	4.2 ± 1.1	70.62 ± 6.7	21.7 ± 6.6	56.24 ± 12.8	51.1 ± 9.7	55.30 ± 3.7	35.5 ± 6.2	43.26 ± 3.1	23.6 ± 2.7
LwF	92.17 ± 1.1	1.5 ± 1.0	86.63 ± 0.5	1.8 ± 0.7	OOM	OOM	95.92 ± 1.7	6.9 ± 2.0	80.21 ± 4.1	12.9 ± 4.8	72.25 ± 7.7	31.5 ± 8.2	72.18 ± 4.2	6.7 ± 3.9	49.33 ± 2.1	16.5 ± 2.0
EWC	90.39 ± 1.9	2.0 ± 1.2	85.43 ± 1.1	2.7 ± 1.2	90.68 ± 2.9	5.7 ± 2.1	97.22 ± 0.8	3.6 ± 1.2	88.52 ± 2.0	3.6 ± 1.7	72.30 ± 5.4	23.9 ± 5.1	71.02 ± 2.9	6.0 ± 2.7	<u>55.47 ± 1.4</u>	6.4 ± 1.7
TWP	90.52 ± 1.3	3.3 ± 2.1	86.07 ± 0.7	2.2 ± 0.9	89.35 ± 3.3	8.9 ± 3.5	97.90 ± 0.6	2.0 ± 0.8	87.45 ± 2.4	3.8 ± 2.1	76.20 ± 7.8	18.2 ± 7.0	72.18 ± 3.7	6.7 ± 3.3	<u>57.25 ± 2.0</u>	6.4 ± 2.2
ERGNN	86.92 ± 4.2	3.7 ± 3.1	86.52 ± 0.7	3.6 ± 1.1	94.39 ± 2.3	3.8 ± 2.7	97.83 ± 0.6	0.5 ± 0.2	89.96 ± 1.3	2.6 ± 1.5	N/A	N/A	N/A	N/A	N/A	N/A
PackNet	93.79 ± 0.7	0.0 ± 0.0	85.66 ± 1.8	0.0 ± 0.0	97.14 ± 0.7	0.0 ± 0.0	98.17 ± 0.2	0.0 ± 0.0	90.93 ± 1.1	0.0 ± 0.0	96.05 ± 1.1	0.0 ± 0.0	<u>80.17 ± 0.7</u>	0.0 ± 0.0	41.08 ± 3.3	0.0 ± 0.0
Piggyback	93.87 ± 0.9	0.0 ± 0.0	87.33 ± 0.4	0.0 ± 0.0	96.39 ± 0.7	0.0 ± 0.0	98.23 ± 0.3	0.0 ± 0.0	<u>92.07 ± 0.5</u>	0.0 ± 0.0	83.12 ± 3.0	0.0 ± 0.0	76.17 ± 1.9	0.0 ± 0.0	41.64 ± 2.7	0.0 ± 0.0
HAT	94.01 ± 1.2	2.4 ± 1.5	85.58 ± 1.2	0.6 ± 0.3	91.79 ± 2.1	7.0 ± 1.4	97.92 ± 0.5	0.4 ± 0.0	90.07 ± 0.4	1.2 ± 0.5	59.91 ± 6.9	46.4 ± 6.8	54.87 ± 4.6	36.3 ± 4.4	54.28 ± 1.9	11.9 ± 2.5
TAAM	94.20 ± 1.1	0.0 ± 0.0	<u>87.99 ± 0.4</u>	0.0 ± 0.0	97.21 ± 1.9	0.0 ± 0.0	<u>98.64 ± 1.1</u>	0.0 ± 0.0	86.27 ± 1.2	0.0 ± 0.0	95.26 ± 1.4	0.0 ± 0.0	70.99 ± 4.2	0.0 ± 0.0	53.51 ± 1.9	0.0 ± 0.0
WSN	94.06 ± 1.1	0.0 ± 0.0	87.38 ± 0.3	0.0 ± 0.0	97.28 ± 0.8	0.0 ± 0.0	98.24 ± 0.2	0.0 ± 0.0	90.28 ± 0.9	0.0 ± 0.0	95.92 ± 1.6	0.0 ± 0.0	80.02 ± 1.7	0.0 ± 0.0	47.15 ± 2.0	0.0 ± 0.0
CAGCN	<u>94.68 ± 0.7</u>	0.0 ± 0.0	87.63 ± 0.3	0.0 ± 0.0	<u>97.32 ± 1.2</u>	0.0 ± 0.0	98.01 ± 0.6	0.0 ± 0.0	<u>91.55 ± 0.7</u>	0.0 ± 0.0	<u>96.50 ± 1.6</u>	0.0 ± 0.0	<u>81.13 ± 1.5</u>	0.0 ± 0.0	49.70 ± 1.1	0.0 ± 0.0
CAGAT	<u>95.28 ± 1.6</u>	0.0 ± 0.0	88.28 ± 0.7	0.0 ± 0.0	<u>97.80 ± 0.9</u>	0.0 ± 0.0	<u>98.32 ± 0.2</u>	0.0 ± 0.0	87.91 ± 1.7	0.0 ± 0.0	<u>96.41 ± 1.1</u>	0.0 ± 0.0	79.81 ± 1.8	0.0 ± 0.0	49.96 ± 1.5	0.0 ± 0.0
Joint	96.02 ± 1.6	0.0 ± 0.0	89.06 ± 2.6	0.0 ± 0.0	98.45 ± 2.3	0.0 ± 0.0	99.09 ± 0.4	0.0 ± 0.0	91.62 ± 2.6	0.0 ± 0.0	97.58 ± 0.7	0.0 ± 0.0	81.65 ± 1.5	0.0 ± 0.0	63.49 ± 3.4	0.0 ± 0.0

WSN (Kang et al., 2024). In particular, we compare its computational efficiency and scalability with WSN, a closely related state-of-the-art element-wise masking method. To assess forgetting mitigation more broadly, we also include TWP (Liu et al., 2021) and ERGNN (Zhou & Cao, 2021) as graph-specific regularization- and replay-based baselines, together with LwF (Li & Hoiem, 2017) and EWC (Kirkpatrick et al., 2017). All methods adopt GCN (Kipf, 2017) as the backbone, except TAAM, which follows its original setting and uses SGC (Wu et al., 2019). We consider a bare GCN without continual learning mechanisms as the lower bound, and joint training (Caruana, 1997), where the model can access all data, as the upper bound in our continual learning setting.

4.2. Performance and Efficiency Analysis

Table 1 reports the Average Performance (AP) and Average Forgetting (AF) of different continual learning paradigms on node and graph classification tasks. Parameter isolation-based methods generally outperform regularization- and replay-based methods, which is expected in the task-incremental setting due to their forget-free characteristic (AF = 0). Within parameter isolation methods, Piggyback and TAAM perform strongly, but their fixed backbone may limit plasticity on some datasets when tasks exhibit substantially different patterns. HAT suffers from forgetting because its used parameters are not fully frozen, especially on CoraFull, which has more tasks than others. WSN and our CAGNN both learn subnetwork structures and network parameters, resulting in comparable performance across datasets. In general, methods with higher plasticity, including PackNet, WSN, and CAGNN, achieve more consistent performance. Overall, CAGNN’s stable results across datasets highlight its effectiveness in mitigating

Table 2. Runtime and memory usage comparison. Runtime is averaged over 10% warm-up epochs and 90% post-warm-up epochs.

(a) Average runtime per epoch						
MODEL	OGB-ARXIV		MNIST		CIFAR10	
	Time (ms)	Ratio	Time (ms)	Ratio	Time (ms)	Ratio
WSN	787	1.00×	9,611	1.00×	10,325	1.00×
PackNet	335	0.43×	6,419	0.67×	6,922	0.67×
Piggyback	299	0.38×	6,831	0.71×	7,114	0.69×
HAT	343	0.44×	6,590	0.68×	7,068	0.68×
CAGNN (0.95)	215	0.27×	3,345	0.35×	2,961	0.29×
CAGNN (0.99)	197	0.25×	3,101	0.32×	2,783	0.27×

(b) Memory usage (in MB) across tasks						
MODEL	$\mathcal{T}_0$	$\mathcal{T}_{35}$	$\mathcal{T}_{70}$	$\mathcal{T}_{105}$	$\mathcal{T}_{140}$	
WSN	1918.5	3744.9	5508.2	7270.7	8980.4	
CAGNN	3121.5	3966.6	4750.2	5558.7	6296.2	

catastrophic forgetting. We further examine the transferability of CAGNN by applying it to Graph Attention Network (GAT) (Veličković et al., 2018). The results show consistent performance across two different GNN backbones, highlighting the generality of CAGNN.

For efficiency analysis, we evaluate in terms of runtime and memory usage. Table 2(a) reports the average training time per epoch and runtime ratio under the same over-parameterized model capacities, with 8,650,752 parameters for OGB-Arxiv and 12,582,912 for MNIST and CIFAR10. We evaluate CAGNN at two sparsity levels, $p = 0.95$ and $p = 0.99$, since its decoupled computation depends on the subnetwork size. WSN is the slowest due to element-wise subnetwork learning and full-network propagation, while PackNet, Piggyback, and HAT benefit from simpler masking mechanisms. In contrast, CAGNN achieves the best efficiency, running about 3.6× faster than WSN

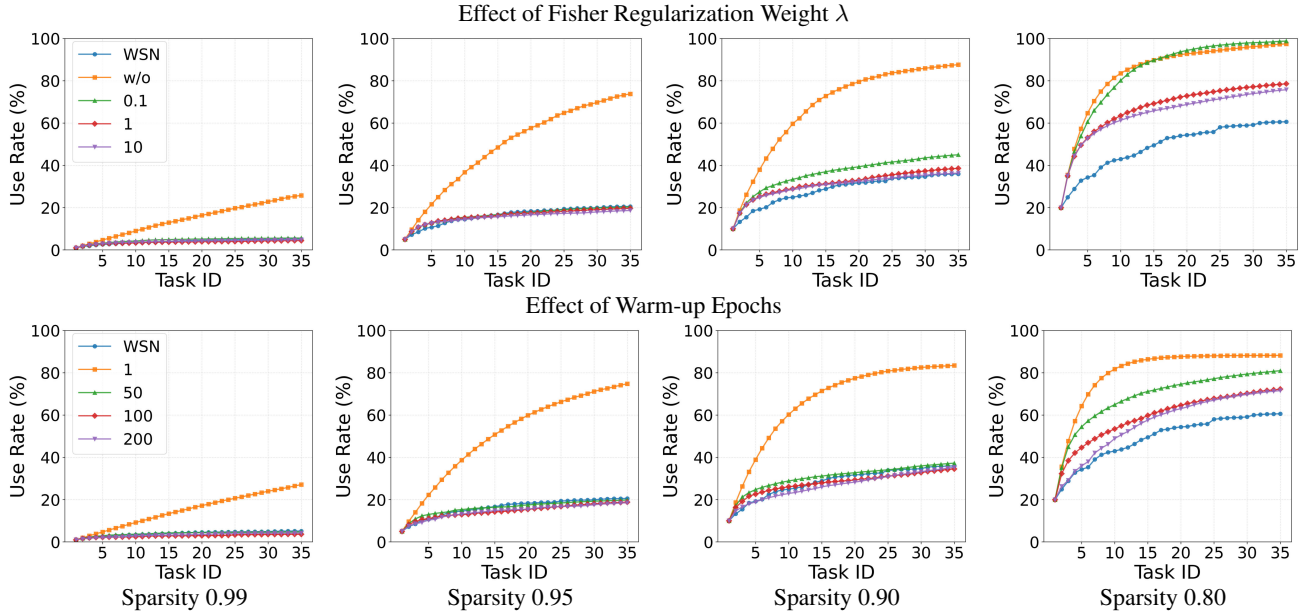


Figure 5. The impact of Fisher regularization weight λ and warm-up epochs on parameter utilization across four sparsity levels. The upper row varies λ with 50 warm-up epochs in 500 total epochs, while the bottom row varies warm-up epochs with $\lambda = 1$.

while maintaining strong plasticity. Moreover, higher sparsity further improves efficiency, and even during warm-up, CAGNN remains faster than WSN, with per-epoch runtimes of 563 ms, 8,849 ms, and 8,468 ms on these three datasets, respectively. Table 2(b) further compares memory usage under the same model capacity across a synthetic CoraFull dataset, which has 140 tasks. As can be seen, CAGNN’s memory growth is much slower than WSN as the number of tasks increases. This demonstrates that CAGNN scales more efficiently in memory usage, owing to its index-form mask storage. These advantages will become more pronounced as the task stream or model capacity grows.

4.3. Scalability and Interference Analysis

Scalability measures a model’s ability to preserve capacity for longer task streams through parameter reuse. We first evaluate it on CoraFull from the perspectives of Fisher regularization and warm-up length. For fair comparison, CAGNN and WSN use the same model capacity. Figure 5 (upper row) shows that Fisher regularization substantially improves parameter utilization. With only 10% warm-up epochs, CAGNN reaches a utilization level close to WSN, which optimizes subnetworks throughout training. Without Fisher regularization, CAGNN consumes parameters faster and is prone to saturation as it re-initializes mask scores for each task. Figure 5 (bottom row) shows that different warm-up lengths yield similar utilization across sparsity levels, suggesting that a short warm-up is sufficient. We further evaluate CAGNN on synthetic CoraFull, which contains 140 tasks for long-horizon continual learning, and study contextual information. Table 3(a) shows that CAGNN maintains robust performance with effective parameter reuse in long-

Table 3. Ablation studies on contextual information and differentiable proxy.

(a) Effect of contextual information (Sparsity 0.99)		
CONTEXT TYPE	SYN-CORAFULL	Usage
Parameterized Score	96.21 $\pm$ 0.7	55.50%
Random Context	95.96 $\pm$ 1.7	57.80%
All 1’s Context	96.51 $\pm$ 1.1	61.30%
Task-specific Context	97.05 $\pm$ 0.3	22.45%
(b) Effect of differentiable proxy (Sparsity 0.90)		
DATASET (# Task)	w/o Diff. Proxy	Usage
CORA (3)	93.97 $\pm$ 1.2	27.10%
CITESEER (3)	86.64 $\pm$ 2.7	25.20%
CORAFULL (35)	95.20 $\pm$ 0.9	83.70%
COMPUTERS (5)	97.72 $\pm$ 0.3	61.50%
OGB-ARXIV (8)	89.81 $\pm$ 1.4	55.10%
MNIST (5)	96.22 $\pm$ 1.2	47.80%
CIFAR10 (5)	77.52 $\pm$ 1.5	48.40%
AROMATICITY (10)	45.43 $\pm$ 2.1	75.30%

task settings. Furthermore, context mainly affects parameter usage, and Fisher regularization becomes less effective when informative task-specific context is absent. Table 3(b) highlights the necessity of the differentiable proxy for mask learning. Although average performance is only slightly affected, gradient truncation makes subnetworks harder to optimize and results in higher parameter consumption. More scalability results are provided in Appendix E.6.4.

We also conduct a case study on CoraFull to validate our theory by evaluating interference under different overlap levels. We first set all model parameters learnable and record each task’s performance as the zero-interference baseline, where training is independent of previous tasks. The interference from frozen reused parameters is then measured by the per-

Table 4. Ablation study of two-stage masking. For each dataset, the same sparsity is used across different stage variants.

VARIANT	CORA (3)		CITESEER (3)		CORAFULL (35)		COMPUTERS (5)		OGB-ARXIV (8)		MNIST (5)		CIFAR10 (5)		AROMATICITY (10)	
	AP	Usage	AP	Usage	AP	Usage	AP	Usage	AP	Usage	AP	Usage	AP	Usage	AP	Usage
Stage 1	93.15±1.3	29.10%	82.63±4.2	80.56%	97.31±0.3	72.20%	95.55±1.2	44.50%	87.37±1.1	58.30%	96.89±1.2	62.80%	82.21±1.3	56.10%	45.61±2.3	94.20%
Stage 2	93.05±1.9	12.70%	83.57±3.2	60.50%	97.18±0.4	96.10%	95.92±1.9	34.70%	89.67±1.4	69.60%	94.46±1.4	21.20%	78.45±1.9	21.50%	43.38±2.0	84.90%
Stage 1&2	94.41±1.6	13.90%	85.64±3.5	62.70%	97.13±0.4	55.30%	97.81±0.4	30.20%	90.97±1.1	48.10%	96.04±1.7	22.10%	81.29±1.1	19.90%	49.28±1.2	60.85%

Table 5. Performance comparison under different overlap ratios.

OVERLAP (SPARSITY)	$\mathcal{T}_5$	$\mathcal{T}_{10}$	$\mathcal{T}_{15}$	$\mathcal{T}_{20}$	$\mathcal{T}_{25}$	$\mathcal{T}_{30}$	$\mathcal{T}_{35}$
0 Interference	95.71	98.67	98.51	88.66	98.27	100.00	99.38
22.8% (0.99 w/o Fisher)	96.32	99.11	98.51	85.57	97.84	100.00	99.38
88.5% (0.99)	95.71	99.11	98.51	87.63	98.70	100.00	99.38
89.7% (0.95)	96.32	98.67	98.51	87.63	98.70	100.00	100.00
90.3% (0.90)	96.32	99.11	98.51	88.66	97.84	100.00	100.00

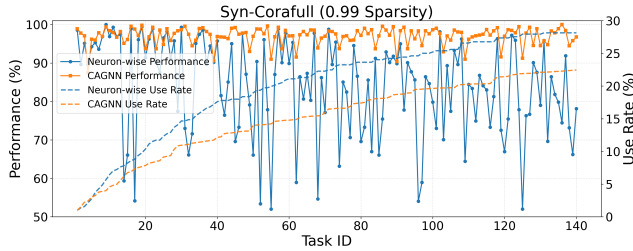


Figure 6. Performance comparison against neuron-wise masking.

formance gap. As shown in Table 5, four sparsity levels with different overlap ratios achieve nearly the same accuracy as the zero-interference baseline across 35 tasks, demonstrating the robustness of our two-stage design to reuse-induced interference. We also compare with neuron-wise masking, an efficiency-oriented scheme corresponding to Proposition 3.1, in Figure 6. Although neuron-wise masking maintains high parameter reuse on the synthetic CoraFull, such reuse does not always lead to beneficial knowledge transfer. When the task sequence becomes longer, strong interference from frozen parameters causes the model to fail to converge on some tasks.

4.4. Ablation and Sensitivity Analysis

The two-stage design is motivated by the limitations of using either stage alone. It aims to achieve fine-grained parameter masking while reducing interference and improving efficiency. Although two stages are designed as a unified masking scheme, we provide an ablation study in Table 4. Removing either stage only slightly affects performance, as each stage can independently function as a parameter-isolation structure. However, parameter reuse drops significantly, demonstrating the necessity of the two-stage design for scalable reuse.

We further conduct sensitivity analysis on CoraFull to evaluate CAGNN’s sensitivity to hidden dimension and branch number, which jointly determine model capacity. As shown in Figure 7, CAGNN remains stable across different capac-

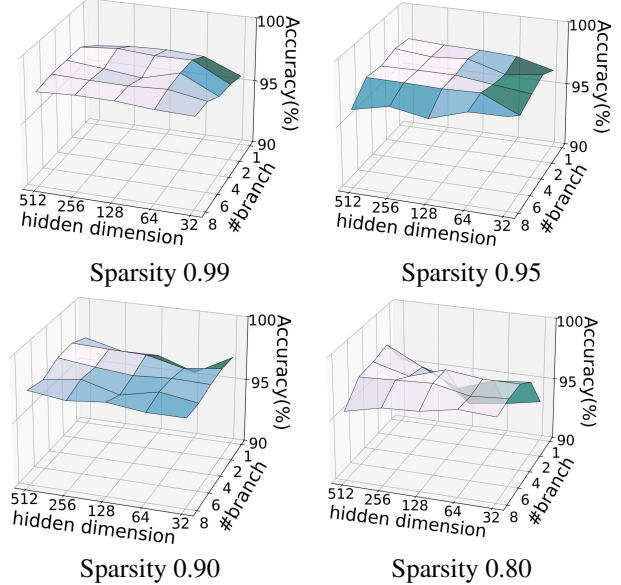


Figure 7. Sensitivity to the number of branches and hidden dimension (model capacity) on the CoraFull dataset.

ities, and sparsity has a limited impact on performance. Lower-capacity settings, such as models with only one branch, show slightly lower performance and larger fluctuations, especially under lower sparsity, suggesting the importance of over-parameterization for stability and scalability. The maximum performance gap across capacity settings is only 2.17%, highlighting CAGNN’s robustness to hyperparameters. More results are provided in Appendix E.6.5.

5. Conclusion

In this work, we identify the efficiency bottleneck of the parameter-isolation paradigm and propose CAGNN, a biological neuron-inspired, capacity-agnostic framework for efficient continual graph learning. CAGNN uses two-stage hierarchical masking to construct task-specific subnetworks from contextual information and decouples them from the full network for efficient training and inference under the capacity expansion scenario. We further introduce Fisher regularization to encourage flexible cross-task parameter reuse. Extensive experiments on the model’s plasticity, efficiency, scalability, and stability show that CAGNN consistently outperforms strong parameter-isolation baselines in both catastrophic forgetting mitigation and computational efficiency.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Achanta, R., Shaji, A., Smith, K., Lucchi, A., Fua, P., and Süsstrunk, S. Slic superpixels compared to state-of-the-art superpixel methods. *IEEE transactions on pattern analysis and machine intelligence*, 34(11):2274–2282, 2012.
- Aghajanyan, A., Gupta, S., and Zettlemoyer, L. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: long papers)*, pp. 7319–7328, 2021.
- Aich, A. Elastic weight consolidation (ewc): Nuts and bolts. *ArXiv*, 2021.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2623–2631, 2019.
- Alon, U. and Yahav, E. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations (ICLR)*, 2021.
- Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *ArXiv*, 2013.
- Bojchevski, A. and Günnemann, S. Deep gaussian embedding of graphs: Unsupervised inductive learning via ranking. In *International Conference on Learning Representations (ICLR)*, 2018.
- Caruana, R. Multitask learning. *Machine learning*, 28(1): 41–75, 1997.
- Chan, K., Stephen, N., and Young, K. Perturbation theory and the rayleigh quotient. *Journal of sound and vibration*, 330(9):2073–2078, 2011.
- Cui, Y., Wang, Y., Sun, Z., Liu, W., Jiang, Y., Han, K., and Hu, W. Lifelong embedding learning and transfer for growing knowledge graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 4217–4224, 2023.
- De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., and Tuytelaars, T. A continual learning survey: Defying forgetting in classification tasks. *IEEE transactions on pattern analysis and machine intelligence*, 44(7):3366–3385, 2021.
- Denton, E. L., Zaremba, W., Bruna, J., LeCun, Y., and Fergus, R. Exploiting linear structure within convolutional networks for efficient evaluation. In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- Dong, X., Zhang, X., and Wang, S. Rayleigh quotient graph neural networks for graph-level anomaly detection. In *International Conference on Learning Representations (ICLR)*, volume 2024, pp. 53937–53955, 2024.
- Dwivedi, V. P., Joshi, C. K., Luu, A. T., Laurent, T., Bengio, Y., and Bresson, X. Benchmarking graph neural networks. *Journal of Machine Learning Research*, 24 (43):1–48, 2023.
- Febrinanto, F. G., Xia, F., Moore, K., Thapa, C., and Aggarwal, C. Graph lifelong learning: A survey. *IEEE Computational Intelligence Magazine*, 18(1):32–51, 2023.
- Fiala, J. C. and Harris, K. M. Dendrite structure. In *Dendrites*, pp. 1–34. Oxford University Press, 1999.
- Frankle, J. and Carbin, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.
- French, R. M. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135, 1999.
- Gao, S., Zhou, M., Wang, Y., Cheng, J., Yachi, H., and Wang, J. Dendritic neuron model with effective learning algorithms for classification, approximation, and prediction. *IEEE transactions on neural networks and learning systems*, 30(2):601–614, 2018.
- Gao, S., Zhou, M., Wang, Z., Sugiyama, D., Cheng, J., Wang, J., and Todo, Y. Fully complex-valued dendritic neuron model. *IEEE transactions on neural networks and learning systems*, 34(4):2105–2118, 2021.
- Golkar, S., Kagan, M., and Cho, K. Continual learning via neural pruning. *ArXiv*, 2019.
- Grewal, K., Forest, J., Cohen, B. P., and Ahmad, S. Going beyond the point neuron: Active dendrites and sparse representations for continual learning. *bioRxiv*, 2021.
- Gunasekar, S., Woodworth, B. E., Bhojanapalli, S., Neyshabur, B., and Srebro, N. Implicit regularization in matrix factorization. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

- Harnett, M. T., Makara, J. K., Spruston, N., Kath, W. L., and Magee, J. C. Synaptic amplification by dendritic spines enhances input cooperativity. *Nature*, 491(7425): 599–602, 2012.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., and Leskovec, J. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems*, volume 33, pp. 22118–22133, 2020.
- Huang, R., Sakamuru, S., Martin, M. T., Reif, D. M., Judson, R. S., Houck, K. A., Casey, W., Hsieh, J.-H., Shockley, K. R., Ceger, P., et al. Profiling of the tox21 10k compound library for agonists and antagonists of the estrogen receptor alpha signaling pathway. *Scientific reports*, 4(1): 5664, 2014.
- Iyer, A., Grewal, K., Velu, A., Souza, L. O., Forest, J., and Ahmad, S. Avoiding catastrophe: Active dendrites enable multi-task learning in dynamic environments. *Frontiers in neurorobotics*, 16:846219, 2022.
- Jang, E., Gu, S., and Poole, B. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations (ICLR)*, 2016.
- Kang, H., Mina, R. J. L., Madjid, S. R. H., Yoon, J., Hasegawa-Johnson, M., Hwang, S. J., and Yoo, C. D. Forget-free continual learning with winning subnetworks. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, pp. 10734–10750. PMLR, 2022.
- Kang, H., Yoon, J., Hwang, S. J., and Yoo, C. D. Continual learning: Forget-free winning subnetworks for video representations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- Kipf, T. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- Ko, J., Kang, S., Kwon, T., Moon, H., and Shin, K. Begin: Extensive benchmark scenarios and an easy-to-use framework for graph continual learning. *ACM Transactions on Intelligent Systems and Technology*, 16(1):1–22, 2025.
- Li, C., Farkhoor, H., Liu, R., and Yosinski, J. Measuring the intrinsic dimension of objective landscapes. In *International Conference on Learning Representations (ICLR)*, 2018.
- Li, R., Xiao, Y., Ma, X., Vasnev, A., and Gao, J. Gdendrite: On heterophilous graph contexts mining with versatile neural dendrites framework. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 1505–1516, 2025.
- Li, Z. and Hoiem, D. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017.
- Liu, H., Yang, Y., and Wang, X. Overcoming catastrophic forgetting in graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 8653–8661, 2021.
- Liu, J. and Zhang, X. Taam: Inductive graph-class incremental learning with task-aware adaptive modulation. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*, pp. 3078–3086, 2026.
- Liu, J., Xia, F., Wang, L., Xu, B., Kong, X., Tong, H., and King, I. Shifu2: A network representation learning based model for advisor-advisee relationship mining. *IEEE Transactions on Knowledge and Data Engineering*, 33(4):1763–1777, 2019.
- Lopez-Paz, D. and Ranzato, M. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Maddison, C. J., Mnih, A., and Teh, Y. W. The concrete distribution: A continuous relaxation of discrete random variables. In *International Conference on Learning Representations (ICLR)*, 2016.
- Mallya, A. and Lazebnik, S. Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7765–7773, 2018.
- Mallya, A., Davis, D., and Lazebnik, S. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 67–82, 2018.
- McAuley, J., Targett, C., Shi, Q., and Van Den Hengel, A. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*, pp. 43–52, 2015.

- Nt, H. and Maehara, T. Revisiting graph neural networks: All we have is low-pass filters. *ArXiv*, 2019.
- Rall, W. Electrophysiology of a dendritic neuron model. *Biophysical journal*, 2(2 Pt 2):145, 1962.
- Ring, M. B. *Continual learning in reinforcement environments*. The University of Texas at Austin, 1994.
- Schachter, M. J., Oesch, N., Smith, R. G., and Taylor, W. R. Dendritic spikes amplify the synaptic signal to enhance detection of motion in a simulation of the direction-selective ganglion cell. *PLoS computational biology*, 6(8):e1000899, 2010.
- Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., and Eliassi-Rad, T. Collective classification in network data. *AI magazine*, 29(3):93–93, 2008.
- Serra, J., Suris, D., Miron, M., and Karatzoglou, A. Overcoming catastrophic forgetting with hard attention to the task. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 4548–4557. PMLR, 2018.
- Todo, Y., Tang, Z., Todo, H., Ji, J., and Yamashita, K. Neurons with multiplicative interactions of nonlinear synapses. *International journal of neural systems*, 29(08):1950012, 2019.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Wang, K., Shen, Z., Huang, C., Wu, C.-H., Dong, Y., and Kanakia, A. Microsoft academic graph: When experts are not enough. *Quantitative Science Studies*, 1(1):396–413, 2020.
- Wang, L., Zhang, X., Su, H., and Zhu, J. A comprehensive survey of continual learning: Theory, method and application. *IEEE transactions on pattern analysis and machine intelligence*, 46(8):5362–5383, 2024.
- Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., and Weinberger, K. Simplifying graph convolutional networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 6861–6871. Pmlr, 2019.
- Wu, X., Liu, X., Li, W., and Wu, Q. Improved expressivity through dendritic neural networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018a.
- Wu, Z., Ramsundar, B., Feinberg, E. N., Gomes, J., Geniesse, C., Pappu, A. S., Leswing, K., and Pande, V. Moleculenet: a benchmark for molecular machine learning. *Chemical science*, 9(2):513–530, 2018b.
- Yuan, Q., Guan, S.-U., Ni, P., Luo, T., Wong, P., Chang, V., and Man, K. L. Continual graph learning: A survey. *Pattern Recognition*, pp. 113600, 2026.
- Zhang, P., Yan, Y., Li, C., Wang, S., Xie, X., Song, G., and Kim, S. Continual learning on dynamic graphs via parameter isolation. In *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*, pp. 601–611, 2023.
- Zhang, X., Song, D., and Tao, D. Hierarchical prototype networks for continual graph representation learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(4):4622–4636, 2022.
- Zhang, X., Song, D., Chen, Y., and Tao, D. Topology-aware embedding memory for continual learning on expanding networks. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 4326–4337, 2024a.
- Zhang, X., Song, D., and Tao, D. Continual learning on graphs: Challenges, solutions, and opportunities. *ArXiv*, 2024b.
- Zhou, F. and Cao, C. Overcoming catastrophic forgetting in graph neural networks with experience replay. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 4714–4722, 2021.

A. Algorithm of CAGNN

Algorithm 1 CAGNN-CHM

Require: Tasks $\{\mathcal{T}_t\}_{t=1}^T$, branch number B , model parameters $\mathbf{W}_0 \in \mathbb{R}^{(F_A+F_B) \times DB}$, subnetwork sparsity p , empty accumulated mask $\mathbf{M}_{accum}$, Fisher factor λ , training iterations E , warm-up iterations E_w ($E_w \ll E$)

- 1: Initialize $\mathbf{W}_0$ into two complementary spaces $\mathbf{W}_A \in \mathbb{R}^{F_A \times D \times B}$ and $\mathbf{W}_B \in \mathbb{R}^{F_B \times D \times B}$
- 2: **for** task $t = 1, \dots, T$ **do**
- 3: Construct task-specific context $\mathbf{C}$ by (11) or (12), and initialize mask learner to calculate $\mathcal{S}_{global}$ and $\mathcal{S}_{local}$
- 4: **for** epoch $e = 1, \dots, E$ **do**
- 5: **if** $e \leq E_w$ **then**
- 6: **Stage 1**
- 7: Calculate branch connection scores $\mathcal{S}$ using self-attention scoring $\mathcal{S}_{global}$
- 8: Generate dendritic mask $\mathbf{M}_{dm}$ following (4)
- 9: Apply (5) to extract global subspaces: $\mathbf{W}_{A_{global}}$ and $\mathbf{W}_{B_{global}}$
- 10: **Stage 2**
- 11: Calculate rank scores s_{local} using self-attention scoring $\mathcal{S}_{local}$
- 12: Calculate r according to p
- 13: Reshape $s_{local} \in \mathbb{R}^{1 \times D} \rightarrow \mathbb{R}^{1 \times r \times k}$
- 14: Perform rank selection following (4) to obtain $\mathbf{M}_{\mathcal{I}}$ and $\mathbf{M}_{CHM}$
- 15: Apply (9) to extract local subspaces: $\mathbf{P}_{soft} \in \mathbb{R}^{F_A \times \tilde{r}}$ and $\mathbf{Q}_{soft} \in \mathbb{R}^{F_B \times \tilde{r}}$
- 16: Reparameterize forward weights $\tilde{\mathbf{W}}_{soft} \leftarrow \mathbf{P}_{soft} \mathbf{Q}_{soft}^\top$
- 17: Forward propagation following (10)
- 18: Compute forward loss $\mathcal{L}$
- 19: **if** $t \neq 1$ **then**
- 20: Compute Fisher regularization term $\mathcal{L}(\mathcal{F})$
- 21: $\mathcal{L} \leftarrow \mathcal{L} + \lambda \mathcal{L}(\mathcal{F})$
- 22: **end if**
- 23: Backward propagation
- 24: **if** $t \neq 1$ **then**
- 25: Freeze $\mathbf{W}_0$ according to $\mathbf{M}_{accum; \mathcal{T}_{t-1}}$
- 26: **end if**
- 27: Update $\mathbf{W}_0$, $\mathcal{S}_{global}$, and $\mathcal{S}_{local}$
- 28: **if** $e = E_w$ **then**
- 29: Record $\mathbf{M}_{dm}$ and $\mathcal{I}$ for $\mathcal{T}_t$
- 30: Accumulate $\mathbf{M}_{accum; \mathcal{T}_t} \leftarrow \mathbf{M}_{accum; \mathcal{T}_{t-1}} \vee \mathbf{M}_{CHM}$
- 31: **end if**
- 32: **else**
- 33: Reconstruct $\tilde{\mathbf{W}}$ using task-specific $\mathbf{M}_{dm}$ and $\mathcal{I}$
- 34: Forward propagation following (8)
- 35: Compute forward loss $\mathcal{L}$
- 36: Backward propagation
- 37: Update $\mathbf{W}_0$
- 38: **end if**
- 39: **end for**
- 40: **end for**

B. Related Works

B.1. Continual Graph Learning

Beyond sequential data streams themselves, learning tasks in many real-world applications often unfold in a sequential manner. Instead of being fully accessible and jointly trainable at the beginning, tasks are usually revealed to the model one by one as new data, new categories, or new scenarios emerge over time. This learning process requires a model to continuously

acquire new knowledge from incoming tasks while preserving the knowledge learned from previous ones. Continual Learning (Ring, 1994; Wang et al., 2024), also known as Incremental Learning, has emerged as a crucial research field to tackle this scenario. A central challenge in Continual Learning is catastrophic forgetting, where a model continually updates its parameters to adapt to new tasks but unintentionally overwrites previously acquired knowledge. As a consequence, the model may achieve good performance on the current task while suffering from severe performance degradation on earlier tasks (French, 1999; Kirkpatrick et al., 2017). This issue becomes particularly important when old data are no longer accessible due to storage, privacy, or practical constraints, making it difficult to simply retrain the model on all historical tasks.

While most existing studies in Continual Learning focus on independent data such as images and text, the same learning paradigm naturally extends to relational data, represented by graphs. Graph-structured data are inherently dynamic in many real-world scenarios, where new types of nodes, associated edges, or even entire graphs may emerge over time. Therefore, graph learning models are also expected to adapt to newly arriving graph tasks without forgetting previously learned structural and semantic knowledge, giving rise to the paradigm of Continual Graph Learning. For instance, in social or citation networks, entities may establish or cease relationships with existing or new entities as the network evolves (Liu et al., 2019; Wang et al., 2020). Similarly, in molecular databases, new types of chemical compounds are gradually collected as experimental techniques advance, and novel structures are identified (Huang et al., 2014; Wu et al., 2018b). These scenarios highlight the necessity of developing continual learning methods that can effectively handle graph-structured data.

B.2. Parameter-Isolation Paradigm on Graphs

Analogous to the principal research streams of Continual Learning, Continual Graph Learning approaches also fall into three broad categories, which are Regularization-based methods, Replay-based methods, and Parameter isolation-based methods (De Lange et al., 2021; Wang et al., 2024; Zhang et al., 2024b; Febrinanto et al., 2023), respectively. Although substantial advances have emerged in recent years (Liu et al., 2021; Zhou & Cao, 2021; Cui et al., 2023; Zhang et al., 2024a; 2023), parameter isolation-based methods on graphs remain significantly underexplored, with far fewer studies in the field compared to the other two (Zhang et al., 2024b; Ko et al., 2025). As the name implies, parameter isolation-based methods tackle catastrophic forgetting by featuring task-specific subnetworks within an over-parameterized global space, where model parameters are isolated for different tasks to different extents. Rather than mitigating catastrophic forgetting, this paradigm guarantees a minimal level of this problem (can be zero percent of forgetting if parameters of previous tasks are completely separated or frozen), showcasing absolute dominance in model performance (Ko et al., 2025).

Nevertheless, there is a trade-off between the effectiveness of models in solving catastrophic forgetting and their space complexity (Zhang et al., 2024b), and many existing works adopt various strategies on parameter sparsity, capacity, and reusability to pursue models’ scalability for future tasks. Specifically, PackNet (Mallya & Lazebnik, 2018) frees capacity by pruning low-magnitude parameters; Winning Subnetworks (WSN) (Kang et al., 2022; 2024) are grounded in the Lottery Ticket Hypothesis (LTH) (Frankle & Carbin, 2019) to identify sparse, potentially overlapping subnetworks through task-specific masks; Parameter Isolation GNN (PI-GNN) (Zhang et al., 2023) progressively rectifies stable information and expands networks for subsequent tasks; Hierarchical Prototype Networks (HPNs) (Zhang et al., 2022) employ Atomic Feature Extractors (AFEs) to construct, select and combine multi-level prototypes, enabling implicit parameter reuse from a prototype viewpoint. Hard Attention to the Task (HAT) (Serra et al., 2018) learns task-specific attention masks over output neurons and protects parameters important for previous tasks. More recently, Task-Aware Adaptive Modulation (TAAM) (Liu & Zhang, 2026) freezes a shared GNN backbone and assigns each task a lightweight Neural Synapse Modulator (NSM), which performs node-attentive adaptive modulation. However, it relies on adding task-specific modulators on top of a frozen backbone, which isolates new knowledge into separate expert modules but does not explicitly promote fine-grained parameter sharing across tasks, leading to linearly increasing task-specific parameters as the task stream grows. Notably, PI-GNN (Zhang et al., 2023) employs a resampling and compression mechanism to mitigate this problem, but this mechanism demands additional training and is not universally applicable to other methods since it was originally designed for time-incremental contexts (dynamic graphs). In task-incremental contexts, which are the main arena for parameter isolation-based methods, PI-GNN is more accurately classified as a hybrid approach.

C. Proof of Theoretical Justification

C.1. Proof of Proposition 3.1

Let columns of the weight be partitioned into a frozen set $\mathcal{I}_f$ and a trainable set $\mathcal{I}_t = \mathcal{I} \setminus \mathcal{I}_f$. The minimum parameter bias from the optimal weight $\mathbf{W}^*$ brought by neuron-wise masking with pure column-wise reuse is limited by an irreducible bias from the not learnable portion $\mathbf{W}^f = \mathbf{W}_{:, \mathcal{I}_f}$, such that:

$$\inf_{\mathbf{W}_{:, \mathcal{I}_t}} \|\mathbf{W}^* - \mathbf{W}\|_F = \|\Delta_{:, \mathcal{I}_f}\|_F \quad (20)$$

Proof. For neuron-wise masking, we have $\mathbf{W}_{:, \mathcal{I}_f}^t = 0$, and thus:

$$\begin{aligned} \|\mathbf{W}^* - \mathbf{W}\|_F &= \|\Delta - \mathbf{W}^t\|_F \\ &= \|\begin{bmatrix} \Delta_{:, \mathcal{I}_t} - \mathbf{W}_{:, \mathcal{I}_t}^t & \Delta_{:, \mathcal{I}_f} - 0 \end{bmatrix}\|_F \\ &= \sqrt{\|\Delta_{:, \mathcal{I}_t} - \mathbf{W}_{:, \mathcal{I}_t}^t\|_F^2 + \|\Delta_{:, \mathcal{I}_f}\|_F^2} \end{aligned} \quad (21)$$

Assuming the trainable set perfectly fits such that $\Delta_{:, \mathcal{I}_t} - \mathbf{W}_{:, \mathcal{I}_t}^t = 0$, the neuron-wise parameter reuse yields an irreducible bias $\|\Delta_{:, \mathcal{I}_f}\|_F$. This finishes the proof. $\square$

C.2. Proof of Theorem 3.2

Let $\mathbf{W} = \mathbf{P}\mathbf{Q}^\top$ be with $\text{rank}(\mathbf{W}) \leq r$. The columns of $\mathbf{P}$ and $\mathbf{Q}$ are partitioned into a frozen set $\mathcal{I}_f$ and a trainable set $\mathcal{I}_t = \mathcal{I} \setminus \mathcal{I}_f$, where $r_t = |\mathcal{I}_t| < r$ denotes the effective trainable rank. Under low-rank masking with column-wise reuse, the minimum reuse-induced bias depends only on the effective trainable rank r_t :

$$\inf_{\text{rank}(\mathbf{W}^t)} \|\mathbf{W}^* - \mathbf{W}\|_F = \|\Delta - \Delta_{(r_t)}\|_F = \left(\sum_{i > r_t}^p \sigma_i^2 \right)^{\frac{1}{2}} \quad (22)$$

where the infimum is taken over the trainable component $\mathbf{W}^t$ with rank at most r_t , $\Delta = \mathbf{U}\Sigma\mathbf{V}^\top$ is the SVD of the reuse bias with singular values $\Sigma = \text{diag}(\sigma_1 \geq \dots \geq \sigma_p \geq 0)$, and $\Delta_{(r_t)} := \mathbf{U}\Sigma_{(r_t)}\mathbf{V}^\top$ is its best rank- r_t approximation.

To prove the above theorem, we introduce two well-known facts as Lemmas without proof.

Lemma C.1 (Unitary Invariance of the Frobenius Norm). *For any orthogonal $\mathbf{U}$, $\mathbf{V}$ and a matrix $\mathbf{P}$: $\|\mathbf{U}\mathbf{P}\mathbf{V}\|_F = \|\mathbf{P}\|_F$.*

Lemma C.2 (von Neumann's Trace Inequality). *For matrices $\mathbf{P}$ and $\mathbf{Q}$ with singular values $\sigma_{\mathbf{P}}$ and $\sigma_{\mathbf{Q}}$ sorted in decreasing order: $\text{trace}(\mathbf{P}^\top \mathbf{Q}) \leq \sum_i \sigma_{\mathbf{P}, i} \sigma_{\mathbf{Q}, i}$, with equality if the singular vectors of $\mathbf{P}$ and $\mathbf{Q}$ are aligned.*

Proof. Without loss of generality, let $\mathbf{P} = [\mathbf{P}_f \ \mathbf{P}_t]$ and $\mathbf{Q} = [\mathbf{Q}_f \ \mathbf{Q}_t]$, where $\mathbf{P}_f = \mathbf{P}_{:, \mathcal{I}_f}$, $\mathbf{Q}_f = \mathbf{Q}_{:, \mathcal{I}_f}$, $\mathbf{P}_t = \mathbf{P}_{:, \mathcal{I}_t}$, and $\mathbf{Q}_t = \mathbf{Q}_{:, \mathcal{I}_t}$. We can rewrite $\mathbf{W} = \mathbf{P}\mathbf{Q}^\top = \mathbf{P}_f\mathbf{Q}_f^\top + \mathbf{P}_t\mathbf{Q}_t^\top$, where $\mathbf{P}_f\mathbf{Q}_f^\top = \mathbf{W}^f$ and $\mathbf{P}_t\mathbf{Q}_t^\top = \mathbf{W}^t$ with rank r_t . Denoting $\Delta = \mathbf{U}\Sigma\mathbf{V}^\top$, we define $\mathcal{W}^t := \mathbf{U}^\top \mathbf{W}^t \mathbf{V}$. By Lemma C.1:

$$\begin{aligned} \|\mathbf{W}^* - \mathbf{W}\|_F &= \|\Delta - \mathbf{W}^t\|_F \\ &= \|\mathbf{U}\Sigma\mathbf{V}^\top - \mathbf{W}^t\|_F \\ &= \|\mathbf{U}^\top(\mathbf{U}\Sigma\mathbf{V}^\top - \mathbf{W}^t)\mathbf{V}\|_F \\ &= \|\Sigma - \mathcal{W}^t\|_F \end{aligned} \quad (23)$$

Let $\sigma_{\mathcal{W}^t}$ be the singular values of $\mathcal{W}^t$, we have $\|\Sigma\|_F^2 = \text{trace}(\Sigma^\top \Sigma) = \sum_{i=1}^r \sigma_i^2$, and $\|\mathcal{W}^t\|_F^2 = \sum_{i=1}^r \sigma_{\mathcal{W}^t, i}^2$. Then, by Lemma C.2:

$$\text{trace}(\Sigma^\top \mathcal{W}^t) \leq \sum_{i=1}^p \sigma_i \sigma_{\mathcal{W}^t, i} \quad (24)$$

Then, we can derive:

$$\begin{aligned} \|\Sigma - \mathcal{W}^t\|_F^2 &= \|\Sigma\|_F^2 + \|\mathcal{W}^t\|_F^2 - 2 \text{trace}(\Sigma^\top \mathcal{W}^t) \\ &\geq \sum_{i=1}^p (\sigma_i^2 + \sigma_{\mathcal{W}^t, i}^2 - 2\sigma_i \sigma_{\mathcal{W}^t, i}) = \sum_{i=1}^p (\sigma_i - \sigma_{\mathcal{W}^t, i})^2 \end{aligned} \quad (25)$$

Because $\text{rank}(\mathcal{W}^t) = r_t$, and $\sigma_{\mathcal{W}^t, i} = 0$ for $i > r_t$, hence:

$$\|\Sigma - \mathcal{W}^t\|_F^2 \geq \sum_{i=1}^{r_t} (\sigma_i - \sigma_{\mathcal{W}^t, i})^2 + \sum_{i>r_t}^p \sigma_i^2 \quad (26)$$

Assuming the trainable set perfectly fits, and for each $i \leq r_t$, the former term is minimized at $\sigma_{\mathcal{W}^t, i} = \sigma_i$, we then have a infimum:

$$\inf \|\Sigma - \mathcal{W}^t\|_F^2 = \|\Delta - \mathbf{W}^t\|_F^2 = \sum_{i>r_t}^p \sigma_i^2 \quad (27)$$

Combining the optimal $\mathbf{W}^{t*} = U_{\Sigma(r_t)} V^\top = \Delta_{(r_t)}$. This completes the proof. $\square$

Remark C.3. Although the upper-level dendritic masking may induce element-wise reuse within $\mathbf{P}$ and $\mathbf{Q}$, the resulting weight can still be decomposed as $\mathbf{W} = \mathbf{P}\mathbf{Q}^\top = \sum_{i=1}^r \mathbf{p}_i \mathbf{q}_i^\top$. In this view, each column pair $(\mathbf{p}_i, \mathbf{q}_i)$ defines a rank-one component that implicitly absorbs the effect of element-wise frozen parameters. These frozen entries can be interpreted as linear constraints imposed on each component $\mathbf{p}_i \mathbf{q}_i^\top$, where the fully non-trainable contributions are incorporated into the frozen part $\mathbf{W}^f$, while the remaining trainable entries provide a constrained correction. Hence, our theoretical analysis serves as a column-wise abstraction of the two-stage masking scheme.

C.3. Proof of Proposition 3.3

Given the optimal parameter θ^* for task $\mathcal{T}_t$, the excess loss $\Delta\mathcal{L} = \mathcal{L}(\theta) - \mathcal{L}(\theta^*)$ can be expressed by:

$$\Delta(\mathcal{L}) \approx \frac{1}{2} \Delta\theta^\top \mathbf{H}(\theta^*) \Delta\theta + o(\|\Delta\theta\|^2) \quad (28)$$

where $\mathbf{H}(\theta^*)$ denotes the Hessian matrix of the loss at θ^* .

Proof. Denoting $\Delta\theta = \theta - \theta^*$, Eq. (15) can be derived using the second-order Taylor expansion on $\mathcal{L}(\theta^* + \Delta\theta)$ at the point θ^* , with $\nabla\mathcal{L}(\theta^*) = 0$. $\square$

D. Complexity Analysis

We analyze the time complexity of CAGNN by comparing it with WSN (Kang et al., 2024) under the same parameter budget, which provides a fair and transparent assessment of computational efficiency. Let N , $|\mathcal{E}|$, F , and B denote the number of nodes, number of edges, hidden dimension, and number of branches, respectively. In this context, CAGNN has total capacity $2BF^2$, and the selected rank is given by $r = (1 - p)BF$, where p is a hyperparameter controlling the parameter sparsity for each subnetwork.

WSN Under the matched capacity setting, the hidden dimension of WSN becomes $F_w = \sqrt{2BF}$. Assuming WSN employs two-phase training (Note that in WSN’s original setting, it optimizes masks throughout training), during warm-up, generating a subnetwork requires top- k selection over F_w^2 mask scores, leading to an additional complexity of approximately $\mathcal{O}(F_w^2)$. Together with the graph convolution cost $\mathcal{O}(NF_w^2 + |\mathcal{E}|F_w)$, the warm-up complexity of WSN is:

$$\mathcal{O}(2BF^2 + 2BNF^2 + \sqrt{2B}|\mathcal{E}|F) \quad (29)$$

After the warm-up, the mask is fixed, and the complexity reduces to:

$$\mathcal{O}(2BNF^2 + \sqrt{2B}|\mathcal{E}|F) \quad (30)$$

CAGNN During warm-up, the mask-learning operations of CAGNN require approximately $\mathcal{O}(NF^2)$ complexity. The low-rank reparameterized transformation $(\mathbf{X}\mathbf{P})\mathbf{Q}^\top$ costs $\mathcal{O}(NFr)$, and the message-passing costs $\mathcal{O}(|\mathcal{E}|F)$. Therefore, the overall warm-up complexity of CAGNN is:

$$\mathcal{O}(NF^2 + NFr + |\mathcal{E}|F) \quad (31)$$

After the warm-up, the mask is fixed, and the complexity reduces to:

$$\mathcal{O}(NFr + |\mathcal{E}|F) \quad (32)$$

Finally, since the selected rank r is typically much smaller than WSN’s hidden dimension, i.e., $r \ll F_w$, CAGNN achieves lower complexity than WSN under a reasonable branch number B . This advantage becomes more pronounced as the model capacity grows, i.e., as F increases. With the introduction of contextual information, we typically make the warm-up phase only account for a small fraction of the training process, making the overall complexity of CAGNN substantially lower than WSN. Moreover, we can further reduce the mask-learning cost by pooling the task-specific context into a vector $\mathbf{c} \in \mathbb{R}^{1 \times F}$. In this case, the warm-up complexity of CAGNN becomes:

$$\mathcal{O}(F^2 + NFr + |\mathcal{E}|F) \quad (33)$$

which is consistently smaller than that of WSN.

E. Supplement of Experiment

E.1. Details of Dataset

Table 6. Summary of Dataset Statistics and Characteristics

	DATASETS	#NODES(GRAPHS)	# EDGES	# ATTRIBUTES	# TASKS	# CLASSES/TASK	# CLASSES
NODE CLASSIFICATION	Cora (Sen et al., 2008)	2,708	10,556	1,433	3	2	7
	Citeseer (Sen et al., 2008)	3,327	9,104	3,703	3	2	6
	CoraFull (Bojchevski & Günnemann, 2018)	19,793	126,842	8,710	35	2	70
	Computers (Hu et al., 2020)	13,752	491,722	767	5	2	10
	OGB-Arxiv (McAuley et al., 2015)	169,343	1,166,243	128	8	5	40
	Syn-CoraFull	~19,793	~126,842	8,710	140	2	280
GRAPH CLASSIFICATION	MNIST (Achanta et al., 2012)	~70.6(55,000)	~564.5	3	5	2	10
	CIFAR10 (Dwivedi et al., 2023)	~117.6(45,000)	~941.2	5	5	2	10
	Aromaticity (Wu et al., 2018b)	~29.7(3,868)	~65.4	2	10	3	30

Cora, Citeseer, and CoraFull are citation networks, where each node represents a scientific publication and its label corresponds to the research field of the publication. Following the task-incremental setting, they are split into binary classification tasks with 3, 3, and 35 tasks, respectively. OGB-Arxiv is also a citation network, where each node denotes a research paper associated with a subject area. It is divided into 8 tasks, with each task containing 5 classes. Amazon Computers is a co-purchase network, where nodes represent products and edges connect items that are frequently bought together. It is organized into 5 tasks with 2 classes per task. We also construct a synthetic CoraFull dataset by repeating the original CoraFull task sequence four times with minor random node dropping, resulting in 140 tasks while preserving the basic properties of the original CoraFull dataset. Note that although tasks are repeated in synthetic CoraFull, each repeated occurrence is regarded as a new task when it arrives in the continual learning stream. MNIST and CIFAR10 are super-pixel graph datasets derived from images, and both are organized into 5 tasks with 2 classes per task. Aromaticity consists of molecular graphs, where nodes represent atoms and edges represent chemical bonds. Its labels indicate the number of aromatic atoms in each molecule. We partition Aromaticity into 10 tasks, with each task containing 3 classes.

E.2. Evaluation Metrics

For the evaluation of the catastrophic forgetting, we use two commonly adopted metrics: Average Performance (AP) and Average Forgetting (AF) (Lopez-Paz & Ranzato, 2017). AP measures the model’s plasticity by calculating the average test performance across all T tasks, and AF measures the model’s stability by calculating the average performance change of each task $\mathcal{T}_t$ after learning T tasks. The following two formulations give them:

$$\text{AP} = \frac{1}{T} \sum_{t=1}^T P_{tt}, \quad \text{AF} = \frac{1}{T-1} \sum_{t=1}^{T-1} (P_{Tt} - P_{tt}) \quad (34)$$

where $P_{i,j}$ records the performance of j th task $\mathcal{T}_j$ after learning i th task $\mathcal{T}_i$ ($i \geq j$). Note that $AF > 0$ indicates forgetting, and each task’s performance on the datasets used in our experiments is measured by accuracy.

For the evaluation of the computational efficiency, we measure the average runtime per epoch after completing all tasks, reported in milliseconds, as well as per-task GPU memory usage, reported in megabytes. Regarding the model’s scalability for future tasks, we evaluate it by the percentage of parameters used after learning all T tasks. To ensure a fair playground for parameter isolation-based methods, we evaluate them under the same budget of the total model’s capacity.

E.3. Hardware and Experimental Framework Setup

We ran all of our experiments on a server equipped with an AMD EPYC 7J13 64-core CPU, 256 GB of RAM, and four NVIDIA GeForce RTX 4090 GPUs, and this configuration was used for all baselines. For all experiments, we run them based on a unified continual graph learning framework, BeGin¹ (Ko et al., 2025), and set all common hyperparameters (e.g., learning rate, number of layers, hidden dimensions, dropout rate) the same across all models during hyperparameter tuning. For hyperparameter tuning, we utilize Optuna² (Akiba et al., 2019), which is a hyperparameter optimization framework, to search for the best model.

E.4. Hyperparameter Search Space

We summarize the hyperparameter search spaces used for CAGNN, WSN, TAAM, the upper-bound model, and the BeGin baselines. All hyperparameters are selected based on the validation performance of each dataset.

Table 7. Hyperparameter search spaces and fixed settings.

MODEL	SEARCH SPACE
CAGNN	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $nhid \in \{16, 32, 64, 128, 256, 512\}$; $branch \in \{1, 2, 3, 4, 5, 6\}$; $dropout \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$; $\alpha \in \{0.1, 0.3, 0.5, 0.8, 1, 2, 5, 10\}$; $sparsity \in \{0.85, 0.90, 0.92, 0.95, 0.97, 0.99\}$; $warm-up\ epochs \in \{1, 25, 50, 100\}$.
BASELINE	
WSN	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $nhid \in \{16, 32, 64, 128, 256, 512\}$; $dropout \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$; $sparsity \in \{0.80, 0.85, 0.90, 0.92, 0.95, 0.97, 0.99\}$.
TAAM	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $nhid \in \{64, 128, 256\}$; $dropout \in \{0.2, 0.4, 0.5, 0.6\}$; $task\ embedding\ dimension \in \{16, 32, 64, 128\}$; $TAAM\ heads \in \{2, 4, 8\}$; $TAAM\ rank \in \{4, 8, 16, 32\}$.
PackNet	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $dropout \in \{0.0, 0.25, 0.5\}$; $weight\ decay \in \{0.0, 5 \times 10^{-4}\}$.
Piggyback	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $dropout \in \{0.0, 0.25, 0.5\}$; $weight\ decay \in \{0.0, 5 \times 10^{-4}\}$; $threshold \in \{10^{-1}, 10^{-2}\}$.
HAT	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $dropout \in \{0.0, 0.25, 0.5\}$; $weight\ decay \in \{0.0, 5 \times 10^{-4}\}$; $fixed\ settings: \lambda = 0.75\ and\ s_{max} = 400.0$.
LwF	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $dropout \in \{0.0, 0.25, 0.5\}$; $weight\ decay \in \{0.0, 5 \times 10^{-4}\}$; $\lambda = 1.0$; $temperature\ T = 2.0$.
EWC	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $dropout \in \{0.0, 0.25, 0.5\}$; $weight\ decay \in \{0.0, 5 \times 10^{-4}\}$; $\lambda = 10000.0$.
TWP	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $dropout \in \{0.0, 0.25, 0.5\}$; $weight\ decay \in \{0.0, 5 \times 10^{-4}\}$; $\lambda_t \in \{100.0, 1000.0\}$; $fixed\ settings: \lambda_l = 10000.0, \beta = 0.01$.
ERGNN	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $dropout \in \{0.0, 0.25, 0.5\}$; $weight\ decay \in \{0.0, 5 \times 10^{-4}\}$; $fixed\ settings: the\ number\ of\ experience\ nodes\ follows\ the\ official\ benchmark\ configuration, sampler\ name\ is\ CM, and\ distance\ threshold\ is\ 0.5$.
Joint	$lr \in \{5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $nhid \in \{16, 32, 64, 128, 256, 512\}$; $dropout \in \{0.2, 0.3, 0.4, 0.5, 0.6, 0.7\}$; $weight\ decay \in \{0, 10^{-5}, 5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}, 10^{-2}\}$.
Bare	$lr \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$; $dropout \in \{0.0, 0.25, 0.5\}$; $weight\ decay \in \{0.0, 5 \times 10^{-4}\}$.

¹<https://github.com/ShinhwanKang/BeGin>

²<https://optuna.org/>

For BeGin baselines, we follow the official benchmark settings. The common search space is: $\text{lr} \in \{10^{-3}, 5 \times 10^{-3}, 10^{-2}, 5 \times 10^{-2}, 10^{-1}\}$, $\text{dropout} \in \{0.0, 0.25, 0.5, 0.7\}$, and $\text{weight decay} \in \{0.0, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}, 10^{-2}\}$. For the default backbone, node-level baselines use 3 layers with hidden dimension 256, while graph-level baselines use 4 layers with hidden dimension 146.

E.5. Data Splitting Procedures

All datasets are evaluated under the Task-IL setting, where task identities are available during inference. We summarize the task construction and train/validation/test splits in Table 8.

Table 8. Data splitting procedures for node-level and graph-level datasets.

DATASET	LEVEL	Task Construction	Train / Val / Test Split
Cora	Node	Based on 6 selected classes from the original 7 classes, constructed into 3 binary classification tasks.	Original citation split: 140 / 500 / 1000 nodes before task grouping.
CiteSeer	Node	Based on 6 classes, constructed into 3 binary classification tasks.	Original citation split: 120 / 500 / 1000 nodes before task grouping.
CoraFull	Node	Based on 70 classes, constructed into 35 binary classification tasks.	Original split provided by the dataset and the BeGin implementation.
Computers	Node	Evaluated under our Task-IL split with 5 tasks.	Split: 40% / 20% / 40%.
OGB-Arxiv	Node	Based on 40 subject areas, divided into 8 tasks.	Official OGB time-based split: training papers published until 2017, validation papers published in 2018, and test papers published since 2019.
MNIST	Graph	Images are converted into super-pixel graphs. The 10 classes are partitioned into 5 tasks.	Original split: 55,000 / 5,000 / 10,000 graphs.
CIFAR10	Graph	Images are converted into super-pixel graphs. The 10 classes are partitioned into 5 tasks.	Original split: 45,000 / 5,000 / 10,000 graphs.
Aromaticity	Graph	Molecules are divided into 30 groups based on labels and organized into 10 tasks.	Original split provided by the released dataset and the BeGin implementation.

E.6. Supplement of Experimental Results

E.6.1. SUPPLEMENTARY ANALYSIS ON EFFICIENCY

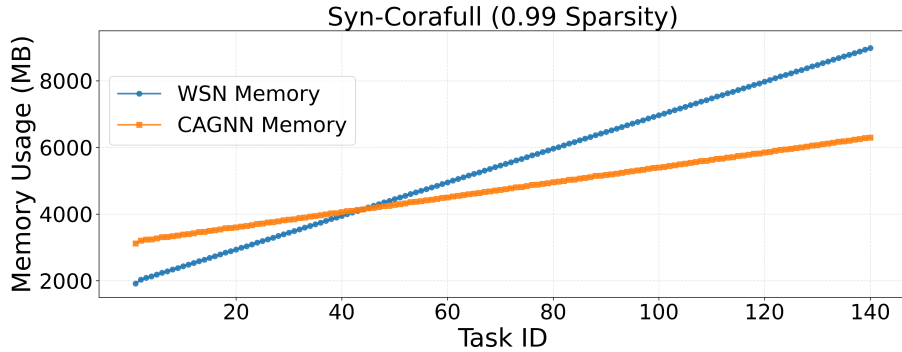


Figure 8. Detailed memory usage of CAGNN and WSN across 140 tasks on the synthetic CoraFull dataset.

Figure 8 visualizes the memory usage of WSN and CAGNN on synthetic CoraFull with 140 tasks under 0.99 sparsity, without any additional compression of the saved masks. Although CAGNN starts with a higher initial memory cost due to its hierarchical masking components, its memory usage grows much more slowly as the task stream becomes longer. In contrast, WSN shows a nearly linear and steeper memory increase, since it needs to maintain dense task-specific masks for an expanding set of tasks. As the number of tasks increases, the memory usage of WSN quickly exceeds that of CAGNN, and the gap becomes increasingly large. This demonstrates that CAGNN’s index-form mask storage provides better memory scalability under continual capacity expansion, making it more suitable for long-horizon continual learning.

E.6.2. SUPPLEMENTARY ANALYSIS ON SCALABILITY

The proposed Fisher regularization substantially improves CAGNN’s scalability with negligible performance degradation, while introducing only a slight increase in warm-up time complexity. This additional cost has already been considered in the previous efficiency analysis. Although the effect is less significant at sparsity levels of 0.99 and 0.95, increasing λ tends to encourage more aggressive parameter reuse across tasks. When the subnetwork sparsity is set to 0.8, CAGNN consumes more parameters than WSN, and a larger λ helps reduce this utilization. Nevertheless, such a low sparsity level is not usually considered, as it accelerates the saturation of model capacity, except when the current sparsity level is insufficient for the current task. Likewise, extending the warm-up phase can further promote parameter reuse, but the improvement becomes marginal and comes with additional training cost, making it less desirable in practice. Notably, Fisher regularization is not applied when the warm-up epoch is set to 1, which leads to high parameter utilization and clear separation among subnetwork structures. Such utilization and separation reveal how task-specific context initially shapes distinct subnetworks, highlighting its importance for effective and efficient subnetwork construction.

We also study a variant where all tasks share the same mask generator, which means that mask scores are not re-initialized for new tasks. Under this setting, contextual information still helps reduce performance degradation while maintaining low parameter utilization, even without Fisher regularization. Nevertheless, this introduces a trade-off between reuse bias and plasticity. Specifically, the shared generator encourages parameter reuse but restricts task-specific adaptation. Consequently, it requires a longer warm-up to reach performance comparable to the original setting, where each task-specific subnetwork is initialized in a more flexible state guided by contextual information. We further attempted WSN’s strategy that does not rely on contextual information or Fisher regularization, where masks are instead initialized as parameterized scores. When the parameterized scores are not re-initialized for each task, the resulting subnetworks become highly similar, leading to low parameter utilization after 50 warm-up epochs. However, such low utilization is largely caused by a nearly fixed subnetwork configuration, where many parameters selected for later tasks have already been frozen and cannot adapt to the current task. This reduces the average performance by approximately 8%. Re-initializing the parameterized scores (see Table 3(a) Parameterized Score) for each task alleviates this issue by improving task-specific adaptability, but at the cost of higher parameter consumption. Recovering performance typically requires a longer warm-up phase, during which the model searches for more suitable subnetworks and mitigates the bias inherited from previous tasks.

E.6.3. SUPPLEMENTARY ANALYSIS ON INTERFERENCE

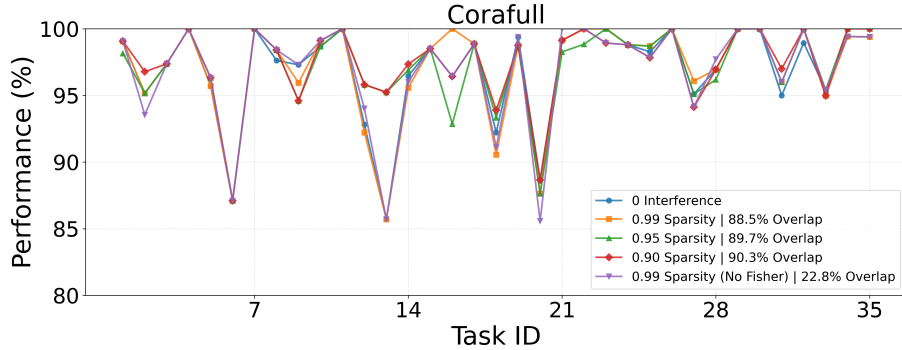


Figure 9. Detailed performance comparison of CAGNN under different overlap ratios on the CoraFull dataset.

Figure 9 provides further performance details of the results in Table 5. We use the fully learnable setting as the zero-interference baseline, where each task is trained without constraints from frozen parameters of previous tasks. As shown in the figure, different sparsity levels with Fisher regularization, i.e., 0.99, 0.95, and 0.90, achieve highly similar task-wise performance to the zero-interference baseline, even though they induce high parameter overlap ratios around 88.5%–90.3%. This indicates that the proposed low-rank masking can tolerate substantial cross-task parameter reuse without introducing notable negative interference. The performance drops observed on several tasks are also shared by the zero-interference baseline, suggesting that they mainly come from task difficulty rather than reuse-induced bias. In contrast, removing Fisher regularization greatly reduces the overlap ratio to 22.8%, showing that Fisher regularization effectively encourages reusable subnetwork structures. Overall, these results support our theoretical claim that the proposed two-stage masking scheme is robust to parameter reuse interference while enabling effective parameter sharing across tasks.

E.6.4. SUPPLEMENT OF SCALABILITY RESULTS

Cora

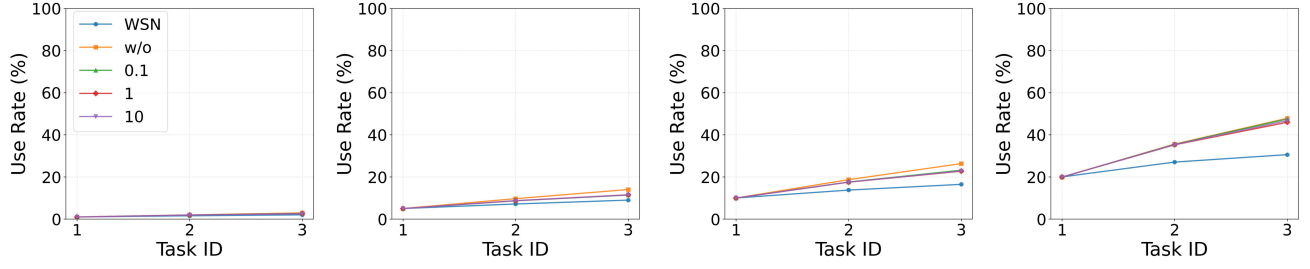


Figure 10. The impact of the Fisher regularization weight λ across 4 different sparsity levels on the Cora dataset (50 warm-up in 500 epochs).

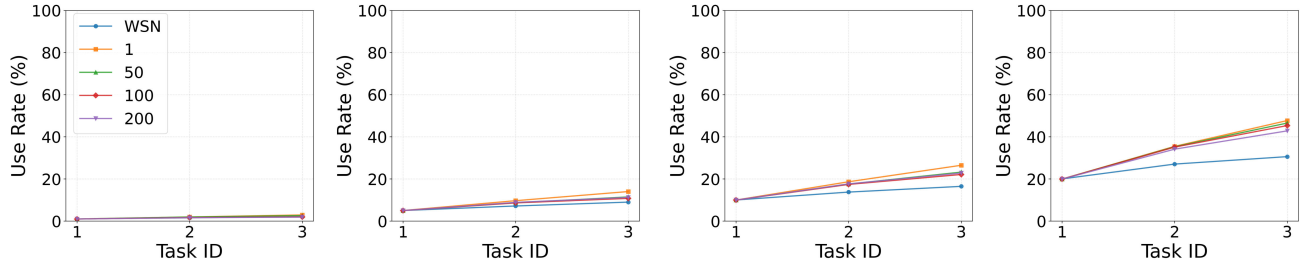


Figure 11. The impact of warm-up epoch across 4 different sparsity levels on the Cora dataset ($\lambda = 1$).

Citeseer

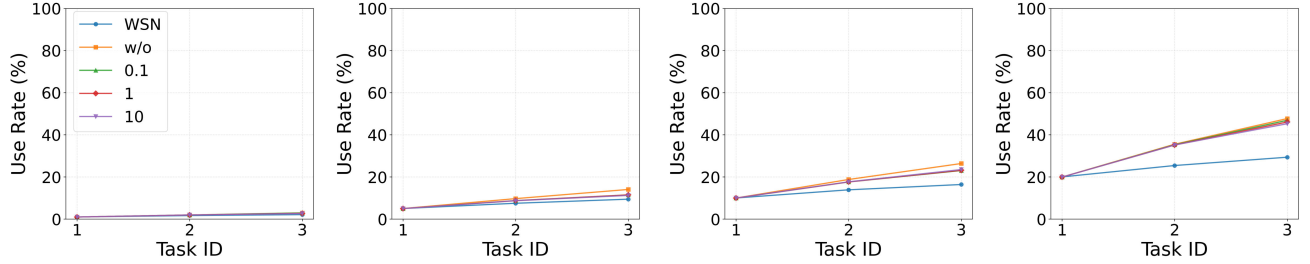


Figure 12. The impact of the Fisher regularization weight λ across 4 different sparsity levels on the Citeseer dataset (50 warm-up in 500 epochs).

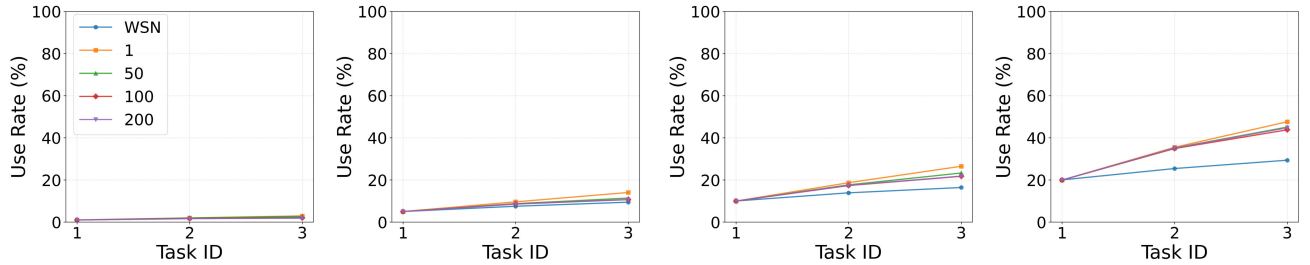


Figure 13. The impact of warm-up epoch across 4 different sparsity levels on the Citeseer dataset ($\lambda = 1$).

Computers

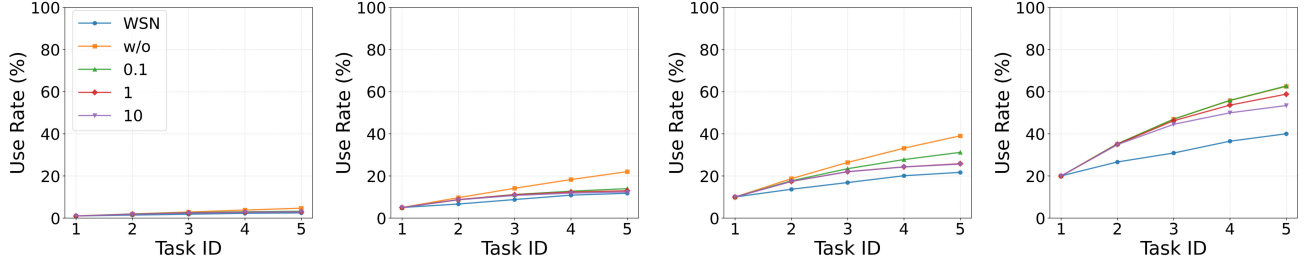


Figure 14. The impact of the Fisher regularization weight λ across 4 different sparsity levels on the Computers dataset (50 warm-up in 500 epochs).

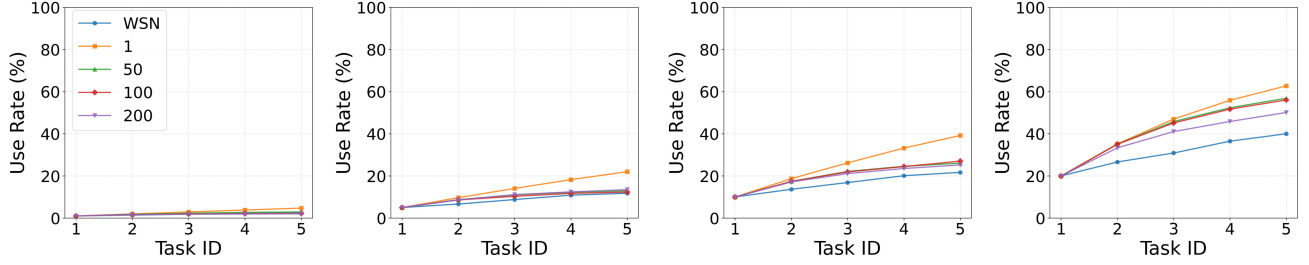


Figure 15. The impact of warm-up epoch across 4 different sparsity levels on the Computers dataset ($\lambda = 1$).

OGB-Arxiv

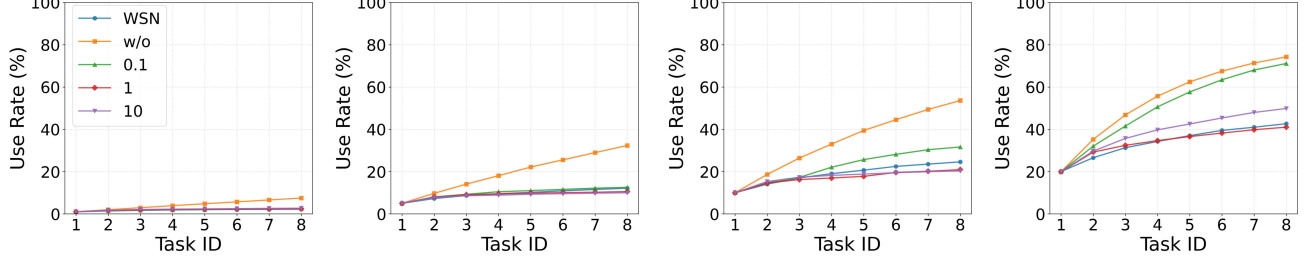


Figure 16. The impact of the Fisher regularization weight λ across 4 different sparsity levels on the OGB-Arxiv dataset (50 warm-up in 500 epochs).

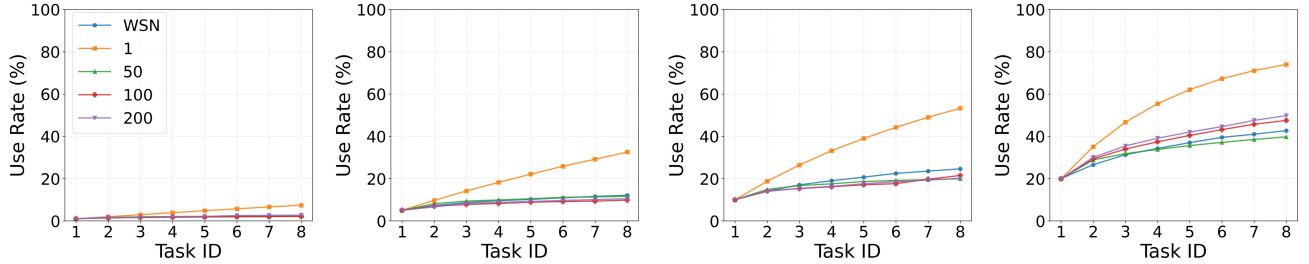


Figure 17. The impact of warm-up epoch across 4 different sparsity levels on the OGB-Arxiv dataset ($\lambda = 1$).

E.6.5. SUPPLEMENT OF SENSITIVITY RESULTS

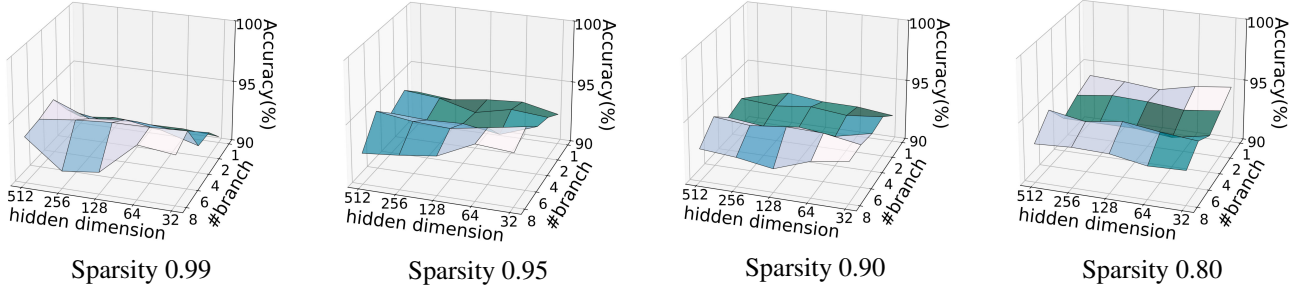
Cora

Figure 18. Sensitivity to the number of branches and hidden dimension (model capacity) on the Cora dataset

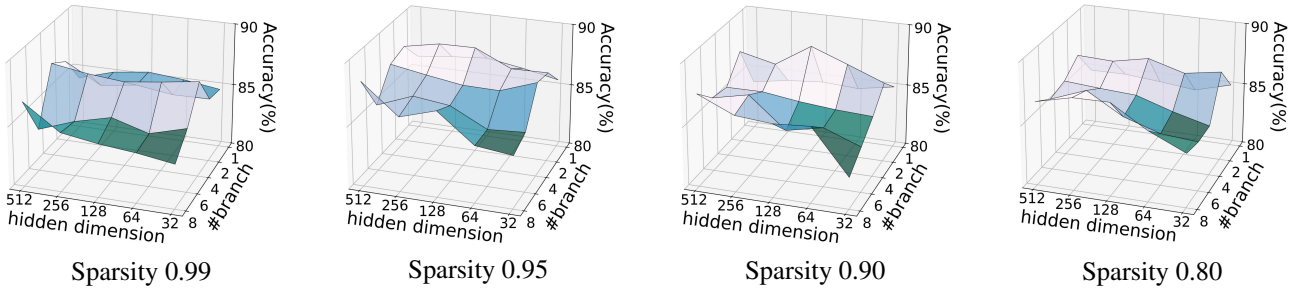
Citeseer

Figure 19. Sensitivity to the number of branches and hidden dimension (model capacity) on the Citeseer dataset

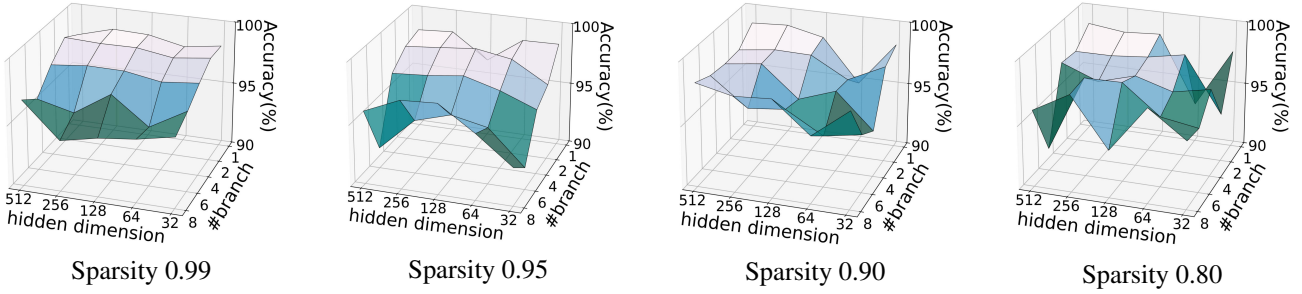
Computers

Figure 20. Sensitivity to the number of branches and hidden dimension (model capacity) on the Computers dataset

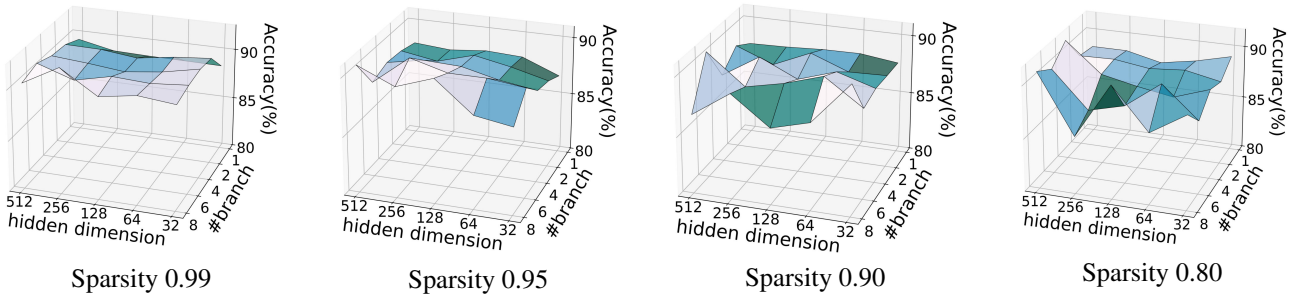
OGB-Arxiv

Figure 21. Sensitivity to the number of branches and hidden dimension (model capacity) on the OGB-Arxiv dataset

E.6.6. VISUALIZATION OF SUBNETWORK STRUCTURES

Task-Specific Context

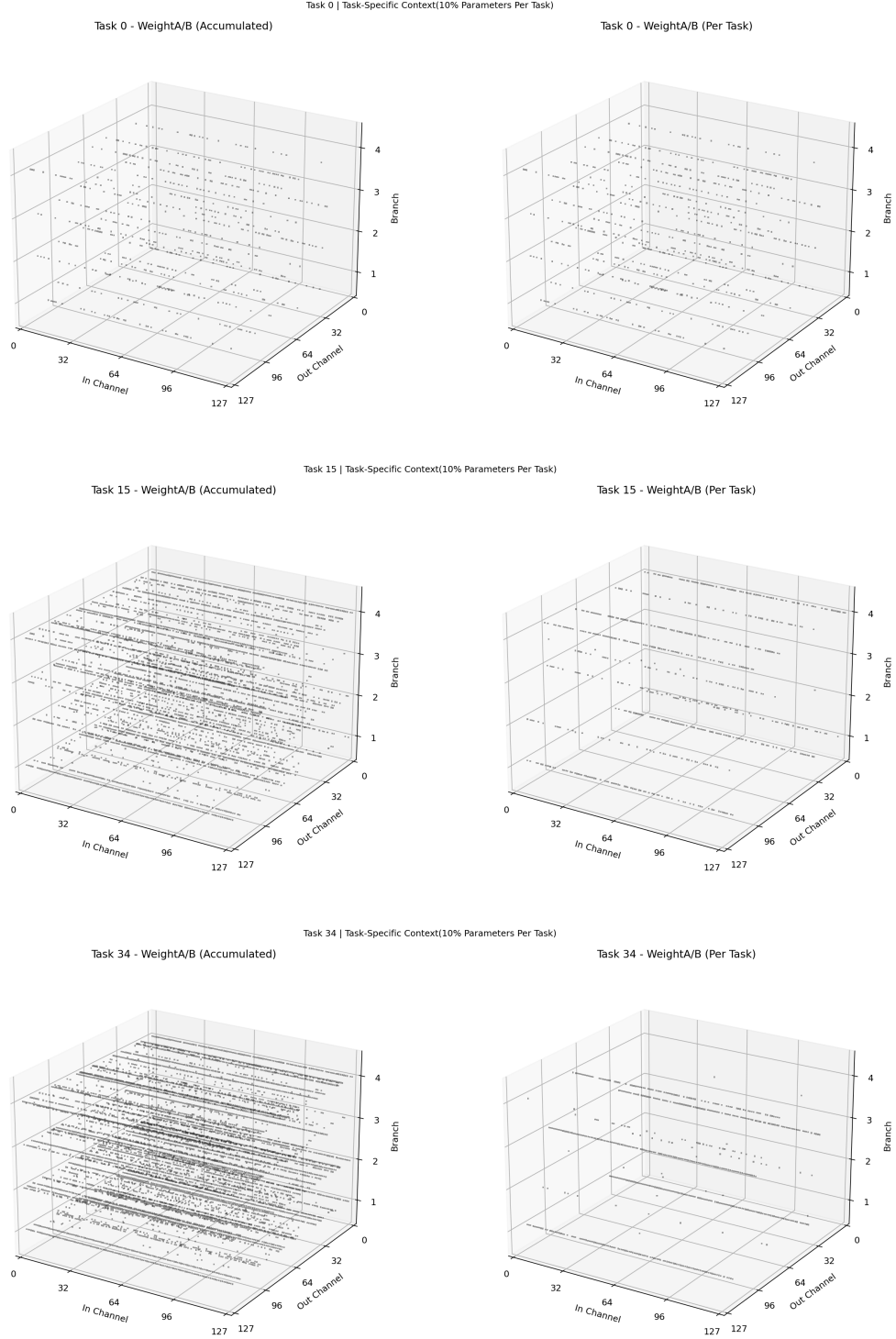


Figure 22. Visualization of subnetwork structures for Task 0, Task 15, and Task 34 in the first convolutional layer on CoraFull, where task-specific context is used for subnetwork construction.

Random Context

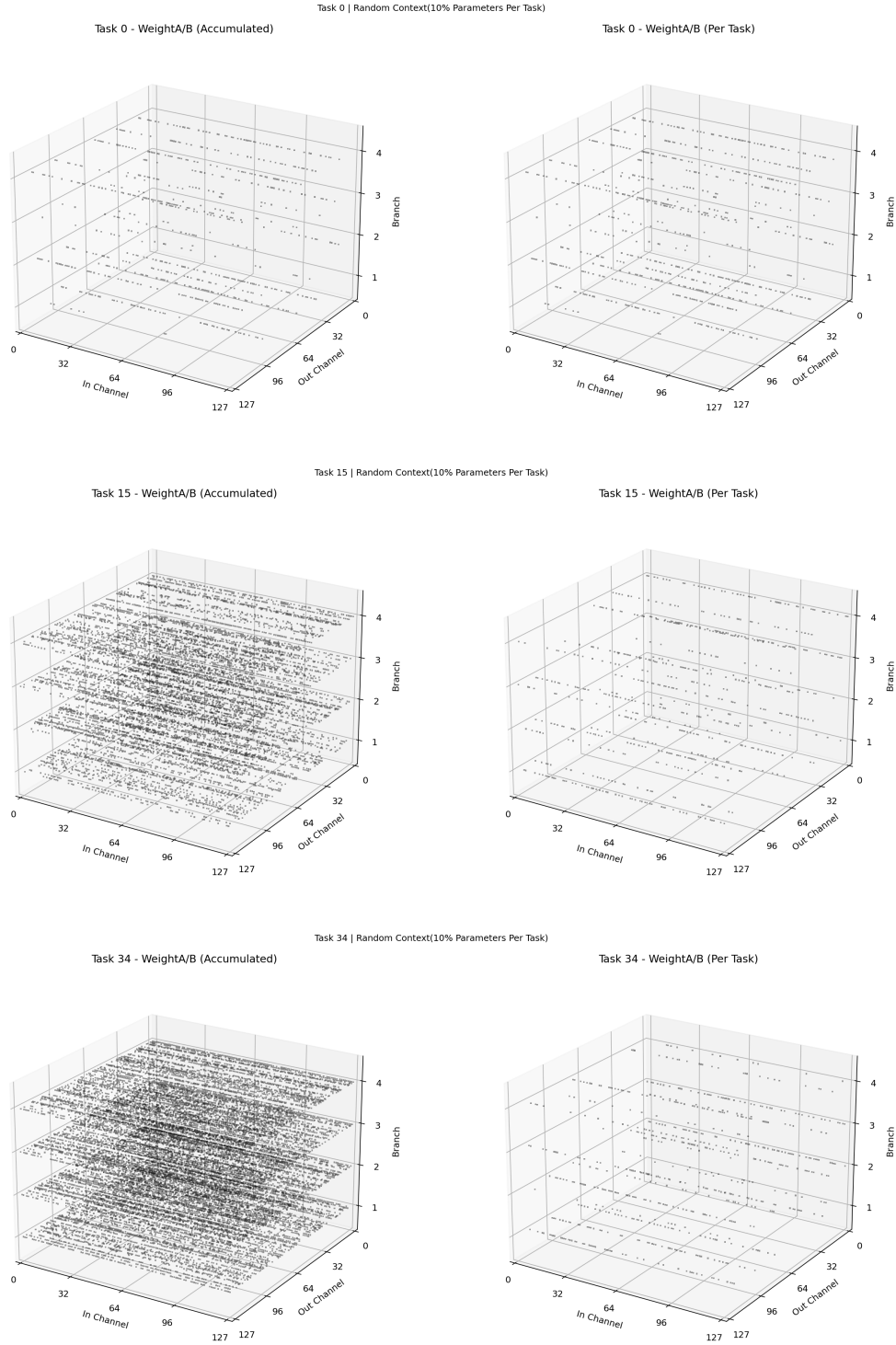


Figure 23. Visualization of subnetwork structures for Task 0, Task 15, and Task 34 in the first convolutional layer on CoraFull, where random context is used for subnetwork construction.